Agritech Answers

Conception d'un système de recommandations pour une agriculture optimisée par les données

Rapport technique

Prototype d'estimation de rendement et de comparaison de marges simulées
Préparation des données · Modélisation · API de prédiction · Interface

Faïza KHELLADI

01/11/2026

Sommaire

Sommaire 2

Agritech Answers — introduction au dossier technique 4

Résumé . . . 4

La demande . . . 4

Les données fournies . . . 4

Les quatre parties . . . 5

Comment lire ce dossier . . . 5

Ce qui ne figure pas ici . . . 5

Étape 1 — Préparation des données : choix réalisés et variables clés 7

1. Ce que contiennent les données fournies . . . 7

2. La décision majeure : il n'y a pas eu de fusion au sens attendu . . . 7

3. Les variables clés du rendement . . . 10

4. Le point dur : la recommandation de culture . . . 12

5. Limites à porter au rapport final . . . 14

6. Ce qui a été livré . . . 14

Étape 2 — Modélisation : performance obtenue et leviers identifiés 15

1. Ce qui a été construit . . . 15

2. Ce que vaut le modèle retenu . . . 15

3. Les leviers du rendement, chiffrés . . . 16

4. La recommandation de culture : la question est tranchée . . . 18

5. La démarche d'optimisation . . . 19

6. Limites à porter au rapport final . . . 20

7. Ce qui a été livré . . . 20

Étape 3 — L'API de prédiction : contrat de service, sourçage et traçabilité 22

1. Le chantier que cette étape devait ouvrir . . . 22

2. Le pont entre ce que l'agriculteur sait et ce que le modèle attend . . . 22

3. Le contrat des six routes . . . 23

4. L'intervalle de prédiction : le protocole de calibration . . . 25

5. Les prix de référence : d'où vient chaque chiffre . . . 26

6. Les charges : pourquoi l'API n'en invente aucune . . . 27

7. Ce que l'API refuse d'affirmer . . . 28

8. La traçabilité, du registre d'expériences au conteneur . . . 28

9. Le conteneur . . . 29
10. Ce qui a été vérifié . . . 30
11. Limites portées à l'étape suivante . . . 31
Étape 4 — L'interface Streamlit : ce que l'agriculteur voit, et ce qu'on l'empêche de croire 32
1. Ce que cette étape devait livrer . . . 32
2. La règle qui gouverne tout le fichier : l'interface ne calcule rien . . . 32
3. Les deux parcours . . . 33
4. Les charges : pourquoi l'écran en propose, et pourquoi il s'en méfie . . . 34
5. La marge brute : la formule, et pourquoi c'est celle-là . . . 35
6. Les champs facultatifs : ce que pèsent vraiment les variables . . . 35
7. L'architecture : deux processus, un seul contrat . . . 37
8. La traçabilité : trois niveaux qu'il serait faux de confondre . . . 37
9. L'export : ce qu'on emporte . . . 38
10. Ce qui a été vérifié . . . 38
11. Limites portées à l'étape suivante . . . 39
Conclusion 40
Ce qui a été livré . . . 40
Ce que le projet établit . . . 40
Ce que le projet n'établit pas . . . 41
Ce qu'il faudrait pour un usage réel . . . 41
Ce que le prototype démontre malgré tout . . . 42

Agritech Answers — introduction au dossier technique

Ce document ouvre le dossier. Les quatre parties qui suivent sont les rapports rédigés à chaque étape du projet, dans l'ordre où ils ont été produits.

Résumé

Agritech Answers estime le rendement d'une parcelle agricole et compare six cultures sur leur marge brute. Huit informations suffisent à l'interroger ; le modèle en attend treize, les cinq autres étant calculées.

Après préparation, le jeu consolidé compte **999 769 parcelles** et 13 variables explicatives, découpées en 70 / 15 / 15. Quatre modèles ont été comparés sur 100 essais d'optimisation chacun. C'est le plus simple qui l'emporte : une **régression régularisée**, retenue pour son erreur, sa rapidité, sa légèreté et la lisibilité de ses coefficients. Sur le jeu de test, ouvert une seule fois après le choix : **0,40 t/ha** d'erreur absolue moyenne, soit 80 €/ha à 200 €/t, et 91 % de la variation du rendement expliquée.

Le résultat central va à l'encontre de la demande. **Les six cultures ne se départagent pas sur le rendement** : à conditions de parcelle identiques, l'écart entre la meilleure et la pire vaut 6,5 kg/ha, quinze fois moins que le seuil de pertinence pratique retenu. Quatre variables portent la totalité du signal — la pluviométrie, puis la fertilisation, l'irrigation et la température ; les neuf autres pèsent quelques kilogrammes par hectare.

Le produit en tire sa forme : une **API de six routes** servant un artefact scellé, et une **interface** qui compare les cultures non sur leur rendement mais sur leur **marge brute**, calculée à partir de prix de référence sourcés et de charges saisies par l'agriculteur.

Une réserve traverse tout le dossier : les données de parcelles sont **simulées**. Le prototype démontre la chaîne complète ; ses chiffres ne sont pas des résultats agronomiques transposables.

La demande

Agritech Answers accompagne des exploitations agricoles dans leurs décisions de culture. La demande portait sur deux questions qu'un agriculteur se pose avant de semer :

- **quel rendement attendre** d'une parcelle, connaissant ses conditions et ses pratiques ;
- **quelle culture choisir** parmi celles qu'il peut implanter.

Le produit attendu est une application utilisable par un agriculteur — pas un modèle dans un notebook. La chaîne complète devait donc être construite : préparation des données, entraînement, mise en service, interface, et l'outillage qui permet de tout rejouer.

Les données fournies

Deux jeux, et une difficulté qui a orienté tout le projet.

Jeu	Volume	Ce qu'une ligne décrit
Parcelles	1 000 000	une parcelle : culture, climat, pratiques, rendement
Référentiel FAO	56 717	un pays, une culture, une année

Les deux ne se joignent pas. Le fichier des parcelles ne contient ni pays ni année ; ses « régions » sont quatre zones génériques sans correspondance géographique. La seule information commune aux deux jeux est le type de culture.

Cette contrainte n'est pas un détail d'implémentation : elle détermine ce que le produit peut affirmer, et ce qu'il doit se refuser à affirmer. La première partie de ce dossier lui est consacrée.

Les quatre parties

Partie	Question traitée	Ce qu'elle établit
1. Préparation des données	que contiennent réellement ces fichiers, et que peut-on en construire ?	un jeu consolidé de 999 769 parcelles et 14 variables, et le constat que les six cultures ne se distinguent pas
2. Modélisation	quel modèle, quelle performance, quels leviers ?	une régression régularisée, 0,40 t/ha d'erreur moyenne, et quatre leviers chiffrés
3. L'API de prédiction	comment servir ce modèle, et avec quelles garanties ?	un service de six routes, son contrat, et ce qu'il refuse d'affirmer
4. L'interface	que voit l'agriculteur, et que l'empêche-t-on de croire ?	deux parcours, et la marge brute qui départage là où le rendement ne départage pas

Chaque partie reprend les limites annoncées par la précédente et dit ce qu'elle en a fait. C'est le fil le plus utile pour suivre le dossier : une limite énoncée à l'étape 1 se retrouve traitée, contournée ou assumée à l'étape 3.

Comment lire ce dossier

Ces documents sont datés. Chacun a été rédigé à la fin de son étape et décrit l'état du projet à ce moment-là. Leurs chiffres — nombre de tests, taille de l'image, périmètre couvert — sont des instantanés justes à leur date, pas l'état actuel. Ils n'ont pas été réécrits après coup : un dossier dont les conclusions bougeraient rétrospectivement ne vaudrait rien.

Les encadrés « Suite donnée » font le lien. Là où une étape ultérieure a levé une limite, un encadré le signale à l'endroit exact où elle était énoncée. Le lecteur n'est donc jamais laissé sur une réserve périmée, sans que le texte d'origine ait été altéré.

Les parties 3 et 4 se lisent à deux niveaux. Leurs premières sections décrivent ce que l'agriculteur voit et ce que l'application affirme — elles se lisent sans connaissance technique. Les dernières portent sur le déploiement, la traçabilité et les tests, et s'adressent à qui reprendra le service.

Ce qui ne figure pas ici

Le rapport métier. Un document distinct, plus court, s'adresse aux agriculteurs : résultats principaux, recommandations agronomiques et explication des variables clés, sans appareil technique. Le présent dossier en est le support de preuve.

L'étape 5 — industrialisation. La chaîne MLOps, l'intégration continue et la livraison de l'image n'ont pas donné lieu à un rapport : elles relèvent du développement, et sont documentées dans le `README.md` du dépôt. On y trouve l'orchestration des quatre étapes de la chaîne, les empreintes qui garantissent la cohérence entre données, modèle et artefact servi, les contrôles exécutés à chaque poussée, et la publication de l'image.

Une réserve qui traverse tout le dossier. Les données de parcelles présentent toutes les caractéristiques d'un jeu **simulé** : variables parfaitement uniformes, modalités rigoureusement équilibrées, relations d'une régularité qu'on n'observe jamais en agriculture réelle. Le prototype démontre la chaîne complète, de la donnée à l'application. Mais les effets chiffrés qu'il contient décrivent le comportement de ce jeu de données, et **ne sont pas des résultats agronomiques transposables** à des parcelles réelles.

Étape 1 — Préparation des données : choix réalisés et variables clés

Document de travail destiné à alimenter le rapport métier final. Rédigé pour un lecteur non technique.

---

1. Ce que contiennent les données fournies

L'archive contient **six fichiers**, et non deux comme annoncé : les deux jeux de données décrits dans la commande, plus les quatre fichiers bruts qui ont servi à construire le second.

Jeu	Volume	Ce qu'une ligne représente
Données de parcelles	1 000 000 lignes	Une parcelle : sa culture, son climat, ses pratiques, son rendement
Référentiel agricole FAO	56 717 lignes	Un pays, une culture, une année

Ces deux ensembles décrivent des réalités différentes. Les parcelles sont des observations individuelles ; le référentiel FAO agrège des moyennes nationales annuelles.

Point déterminant pour la suite : le fichier des parcelles ne contient ni pays ni année. Ses « régions » sont quatre zones génériques — Nord, Sud, Est, Ouest — sans correspondance avec un territoire réel.

---

2. La décision majeure : il n'y a pas eu de fusion au sens attendu

Pourquoi une fusion classique était impossible

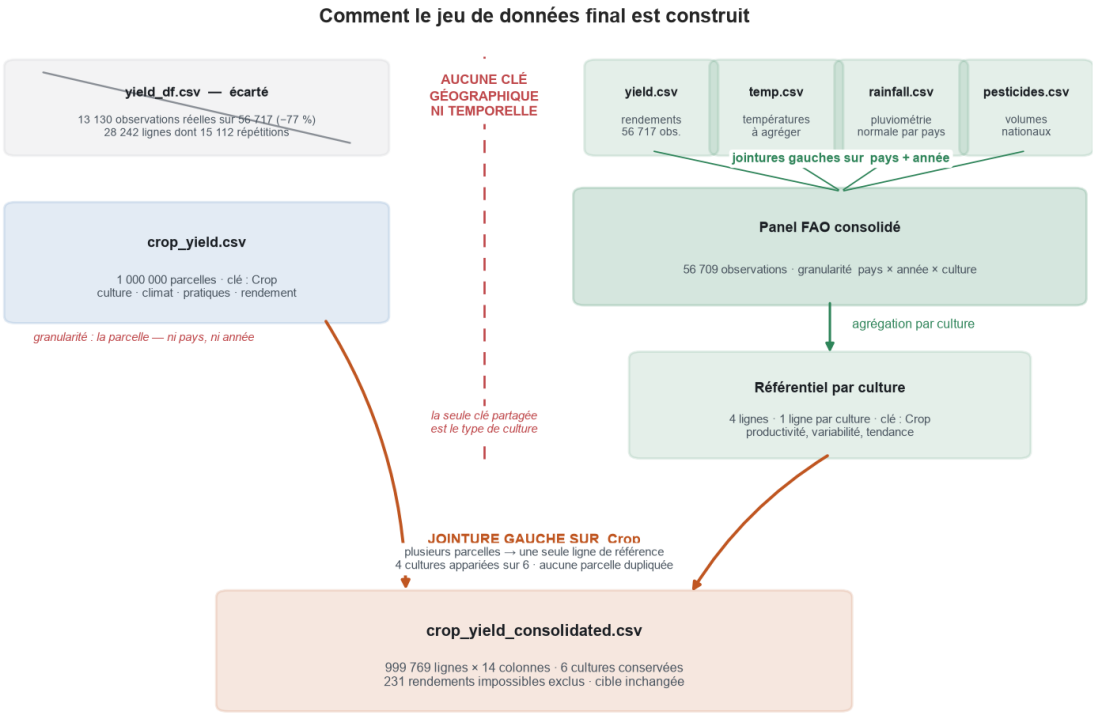
Rapprocher deux jeux de données suppose un identifiant commun. Ici, il n'y en a aucun : les parcelles n'ont ni localisation ni date, le référentiel FAO est organisé par pays et par année. Aucune parcelle ne peut donc être rattachée à un pays ou à une année particulière.

La seule information partagée est le type de culture.

Ce qui a été fait à la place

Nous avons construit un **référentiel par culture** à partir des données FAO — productivité relative, niveau de rendement de référence, variabilité entre pays, progression historique — puis nous l'avons rattaché à chaque parcelle via sa culture.

C'est un **enrichissement documentaire**, pas une fusion géographique. La distinction est importante : une parcelle enrichie ne porte aucune information sur un pays ou une année. Présenter l'opération comme une fusion laisserait croire à une précision qui n'existe pas.



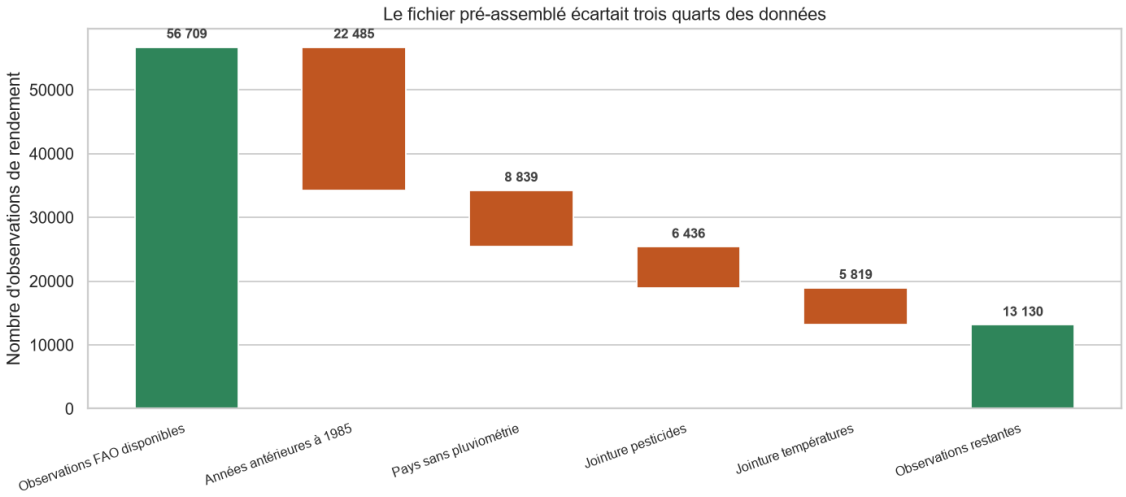
*Le trait rouge sépare deux mondes qui ne partagent **ni pays ni année**. À droite, les quatre sources FAO se combinent normalement sur « pays + année », puis sont agrégées en une table de quatre lignes — une par culture. À gauche, le million de parcelles. La seule clé partagée étant le type de culture, les deux branches convergent par une **jointure gauche sur** `Crop` : plusieurs parcelles pointent vers une même ligne de référence, aucune n'est dupliquée. Le résultat est le fichier unique livré en bas.*

Un choix de source qui change les résultats

La commande désignait un fichier pré-assemblé par le fournisseur des données. Nous avons vérifié comment il avait été construit, et nous ne l'avons **pas retenu**. Trois raisons :

Constat	Conséquence
Sa construction a écarté **77 % des observations de rendement disponibles**	Couverture réduite de 212 à 101 pays
Plus de la moitié de ses lignes sont des répétitions	28 242 lignes pour 13 130 observations réelles
Les répétitions sont **très inégales selon les pays**	L'Inde représente 14 % des lignes pour 1,4 % des observations réelles

La première cause de ces pertes est une simple question de calendrier : les rendements FAO commencent en 1961, les relevés de pluviométrie en 1985. Près de 40 % des observations ont disparu pour cette seule raison — au profit de deux variables dont l'exploitation reste très limitée : la pluviométrie fournie est une moyenne figée par pays, identique d'une année à l'autre, et les pesticides sont un volume national sans rapport avec une dose appliquée à une parcelle.



Chaque barre orange est une jointure qui élimine des observations de rendement. Aucune n'est due à une donnée de mauvaise qualité : ce sont des exclusions par construction, faute de correspondance dans les fichiers annexes.

En repartant du fichier source plutôt que de sa version assemblée, nous disposons de **3,9 fois plus d'observations**, du double de pays et de 56 années d'historique au lieu de 24.

Un piège statistique évité

En comparant les cultures entre elles, un classement établi sur les moyennes mondiales plaçait le blé devant le maïs. Or, **au sein de chaque pays cultivant les deux, c'est l'inverse**.

L'explication est une question de composition : le blé est davantage cultivé dans des pays à agriculture intensive, ce qui gonfle sa moyenne mondiale sans que la culture soit intrinsèquement plus productive. Notre indice est donc calculé **à l'intérieur de chaque pays**, ce qui réduit ce biais — sans toutefois l'annuler complètement, les périodes et les systèmes agricoles restant hétérogènes.

Les variables ajoutées

Cinq variables ont été créées et figurent au jeu de données livré :

Variable	Ce qu'elle apporte
Indice de productivité par culture	Hiérarchie des cultures, corrigée du biais de composition
Variabilité entre pays	Régularité du rendement d'une culture selon les pays
Progression historique	Tendance du rendement sur six décennies
Présence au référentiel	Signale les cultures sans équivalent FAO
Faible pluie sans irrigation	Repère les parcelles en situation défavorable

Quatre autres ont été construites puis **écartées** : deux comparaient des grandeurs climatiques non comparables entre les deux sources, deux faisaient double emploi avec celles retenues. Elles restent disponibles dans le référentiel, sans encombrer le jeu d'entraînement.

Une sixième — un **indice d'intensification** résumant fertilisation et irrigation sur une échelle de 0 à 2 — a été créée puis retirée du livrable. Elle valait exactement la somme des deux pratiques, et aurait empêché d'attribuer un effet à l'une plutôt qu'à l'autre (voir ci-dessous).

Deux vérifications avant de livrer

Aucune valeur extrême n'a été supprimée, et c'est un résultat. Nous avons appliqué une recherche systématique de valeurs aberrantes à toutes les variables quantitatives des cinq fichiers. Elle ne signale rien — pour une raison structurelle : la pluviométrie et la température sont **parfaitement uniformes** (leurs déciles sont réguliers à 0,3 % près), si bien que les seuils de détection tombent en dehors même des valeurs possibles. Quant aux 28 rendements les plus élevés, ils sont exactement en nombre attendu pour un aléa normal — 2 660 observations situées à plus de trois écarts-types de la moyenne, contre 2 699 attendues — et aucune n'a été retirée. **Ce sont des valeurs rares, pas des erreurs** : les retirer aurait biaisé le modèle vers la moyenne.

Deux colonnes ont été retirées, chacune pour un motif précis. Nous avons mesuré la redondance sur l'ensemble des variables numériques, puis étendu l'analyse aux variables qualitatives — nécessaire, car le type de culture détermine entièrement quatre des colonnes construites, ce qu'une analyse purement numérique ne voit pas.

Colonne retirée	Motif
Indice d'intensification	Doublon exact de la fertilisation et de l'irrigation réunies
Durée jusqu'à récolte	Probablement inconnue de l'agriculteur au moment où il interroge l'application, pour un pouvoir explicatif nul

Toutes les autres ont été conservées : leur utilité réelle a été **mesurée à l'entraînement**, à l'étape suivante, plutôt que présumée ici.

---

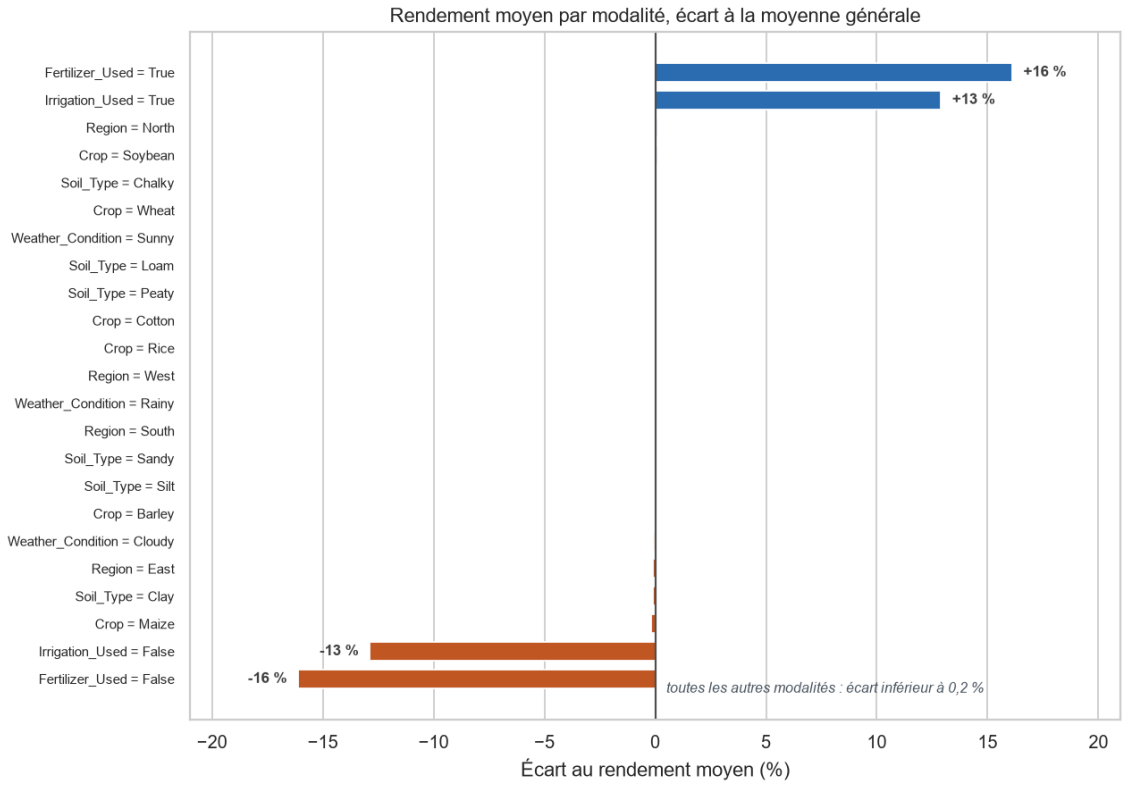
3. Les variables clés du rendement

Ce qui détermine réellement le rendement

Quatre variables concentrent l'essentiel de ce qui est associé au rendement dans les données de parcelles :

Variable	Effet mesuré
Pluviométrie	Le facteur le plus puissant. L'effet est régulier : chaque tranche de pluie supplémentaire apporte le même gain
Fertilisation	**+16 % de rendement** par rapport à une parcelle non fertilisée
Irrigation	**+13 % de rendement**
Température	Effet réel mais faible

Les deux pratiques agricoles sont les seuls **leviers actionnables** par un agriculteur : ce sont elles qui porteront le message métier de l'application.



Quatre barres se détachent — fertiliser et irriguer, dans un sens comme dans l'autre. Les dix-neuf autres situations sont collées à zéro : ni le sol, ni la région, ni la météo, ni la culture ne déplacent le rendement de plus de 0,2 %. Les barres « oui » et « non » d'une même pratique sont les deux faces d'un même constat, non deux effets distincts.

Peut-on en conclure que fertiliser *fait* gagner 16 % ? **Normalement non : un écart observé n'est pas une preuve d'effet, car les parcelles fertilisées pourraient différer par ailleurs. Nous l'avons vérifié — et dans ce jeu de données, elles ne diffèrent pas : entre parcelles fertilisées et non fertilisées, la pluviométrie, la température et la durée de cycle sont identiques à moins de 0,13 %** près. Tout indique une attribution aléatoire des pratiques, comme dans un essai comparatif. La lecture en termes de levier est donc défendable ici, ce qu'elle ne serait pas sur des données de terrain.

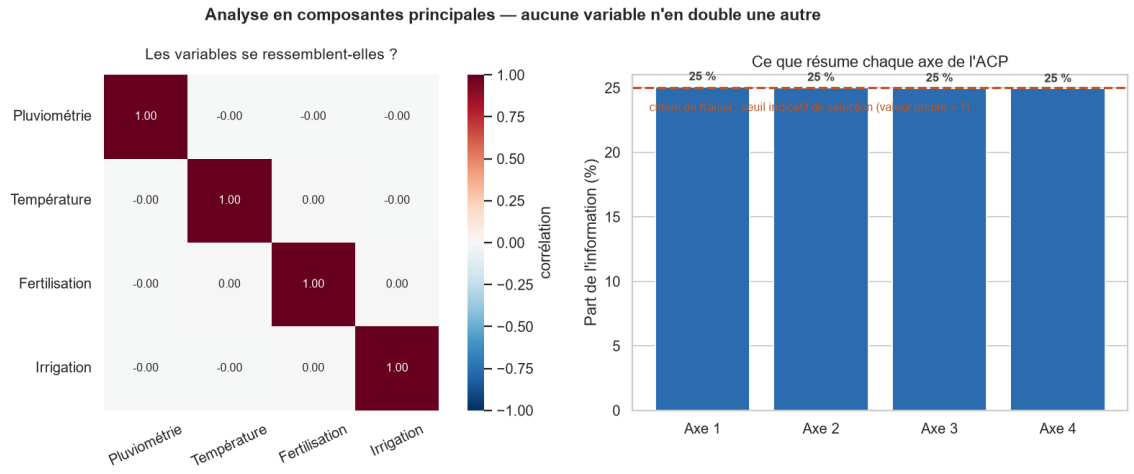
Ce qui n'explique rien — et c'est vérifié

Variable	Constat
Type de sol	Aucun écart mesurable entre les six types
Région	Aucun écart entre les quatre zones
Condition météo	Aucun écart entre ensoleillé, nuageux et pluvieux
Durée jusqu'à récolte	Aucun lien avec le rendement
Culture	**Aucun écart entre les six cultures**

Ces constats ne reposent pas sur une impression : avec un million d'observations, le moindre écart réel aurait été détecté. Les différences observées entre cultures sont de l'ordre de 0,01 tonne par hectare, soit exactement ce que produit le hasard d'échantillonnage.

L'analyse en composantes principales (ACP)

Une question restait ouverte : certaines variables **disent-elles la même chose** sous deux noms différents ? Si c'était le cas, on pourrait les remplacer par un indicateur unique et simplifier le modèle. C'est à cela que sert une ACP.



À gauche, la ressemblance entre variables, deux à deux. Seule la diagonale est colorée — c'est-à-dire que chaque variable ne ressemble qu'à elle-même. Toutes les autres cases valent zéro. À droite, l'ACP cherche à résumer les cinq variables en un plus petit nombre d'indicateurs : elle échoue, chaque axe ne captant qu'un cinquième de l'information, soit exactement sa part si rien n'était regroupable.

Conclusion de l'ACP : aucune redondance linéaire notable parmi les cinq variables construites. Chacune apporte une information qui lui est propre, et il n'existe pas de résumé plus compact de ces cinq-là. L'analyse ne porte pas sur les treize variables du modèle — dont quatre sont entièrement déterminées par la culture ; c'est le contrôle qualitatif décrit plus bas qui traite ce cas.

Une ACP ne détecte toutefois que les ressemblances **linéaires**. Nous avons donc complété par un contrôle direct : un modèle capable de capter n'importe quelle forme de relation a tenté de reconstituer chaque variable à partir des quatre autres. **Il échoue sur les cinq**, avec des performances nulles ou négatives. L'absence de redondance vaut donc aussi hors du linéaire, et les quatre variables clés doivent être conservées telles quelles.

Un point important sur la portée de cette analyse : **l'ACP ne regarde pas le rendement**. Elle compare les variables entre elles, pas leur lien avec ce qu'on cherche à prédire. Elle ne peut donc pas désigner à elle seule les variables importantes — elle confirme que celles identifiées plus haut ne se recoupent pas.

Une réserve sur l'origine des données

Les données de parcelles présentent toutes les caractéristiques d'un jeu **simulé** et non collecté sur le terrain : variables parfaitement uniformes, modalités rigoureusement équilibrées, relations d'une régularité qu'on n'observe jamais en agriculture réelle.

Cela ne remet pas en cause le prototype — il démontrera la chaîne complète, de la donnée à l'application. Mais les effets chiffrés ci-dessus décrivent le comportement de ce jeu de données, et **ne doivent pas être présentés comme des résultats agronomiques** transposables à des parcelles réelles.

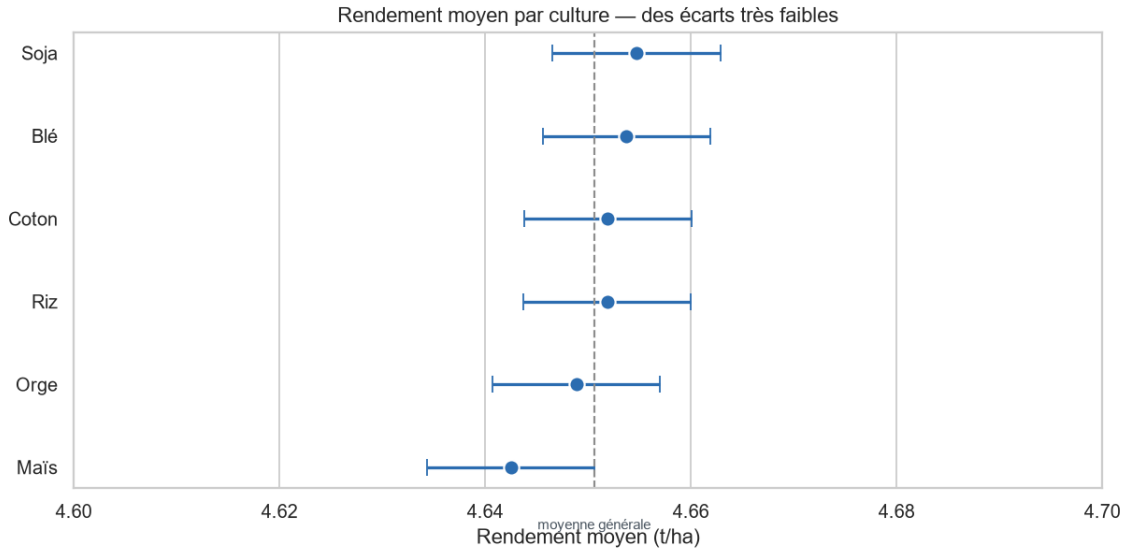
---

4. Le point dur : la recommandation de culture

Le problème

L'application doit remplir deux fonctions. La première — estimer le rendement d'une culture dans des conditions données — ne pose pas de difficulté.

La seconde — **classer les cultures de la plus intéressante à la moins intéressante** — se heurte à un obstacle de fond : les données d'entraînement ne font aucune différence entre les cultures. Simuler les six cultures pour une même parcelle produirait six résultats identiques, et le classement affiché ne serait que du bruit.



Attention à l'échelle : elle couvre un dixième de tonne par hectare. Les six cultures tiennent dans cet intervalle minuscule et leurs marges d'incertitude se recouvrent toutes — aucune hiérarchie fiable ne peut en être tirée.

Nous avons cherché un effet caché de plusieurs manières : à type de sol fixé, à météo fixée, à région fixée, à pratique fixée, et par tranche de pluviométrie ou de température. **Aucun n'apparaît**, y compris avec un test statistique spécifiquement construit pour en détecter un.

Nous avons également comparé la performance d'un modèle qui mémoriserait chaque combinaison de conditions, avec et sans la culture : l'ajouter **dégrade** le résultat. Aucun des modèles et tests employés ne met en évidence d'effet exploitable de la culture. Ce n'est pas une démonstration mathématique qu'aucun modèle au monde n'y parviendrait, mais le faisceau d'indices est serré — et à ce stade, rien ne permet de fonder un classement fiable.

Les deux voies possibles

Ce choix est **métier avant d'être technique**, et il doit être tranché avant la mise en production :

Option A — injecter une référence externe. Utiliser le référentiel FAO pour différencier les cultures, en assumant que la hiérarchie observée au niveau mondial s'applique aux parcelles. L'application afficherait alors un classement cohérent, mais fondé sur une **hypothèse de transfert** entre données nationales historiques et parcelles individuelles — hypothèse qu'il faudrait évaluer et signaler à l'utilisateur.

Option B — renoncer au classement différencié. Présenter l'application comme un estimateur de rendement, en indiquant que les données disponibles ne permettent pas de départager les cultures.

À ce stade, aucune des deux n'a été appliquée. Le fichier livré contient les rendements exactement tels qu'ils figurent dans les données d'origine, sans aucune correction. Ce choix reste entier pour l'étape de modélisation.

> **Suite donnée.** L'étape 2 a tranché pour l'**option B**, sur deux résultats. Retirer l'information « culture » du modèle ne dégrade pas la prédiction : aucun gain prédictif n'est détecté. Et l'écart de rendement entre cultures, à conditions de parcelle identiques, vaut 0,0065 t/ha — soit 6,5 kg/ha, quinze fois moins que le seuil de pertinence pratique de 0,1 t/ha retenu pour le prototype. Le détail figure au rapport de modélisation, section 4.

5. Limites à porter au rapport final

Le classement sera un classement de rendement, pas de rentabilité. La commande parle de « la culture la plus rentable ». La rentabilité dépend du prix de vente, des coûts de semences, d'intrants, d'irrigation et de traitement — **aucune de ces données n'a été fournie**. L'application pourra dire quelle culture produit le plus de tonnes à l'hectare ; elle ne pourra pas dire laquelle rapporte le plus. Obtenir des données économiques, même approximatives, changerait la valeur du produit.

> **Suite donnée.** Les étapes 3 et 4 ont levé cette limite, sans que les données manquantes aient été fournies pour autant. Les **prix** viennent d'une source externe — FAOSTAT 2024, prix à la production, médiane et dispersion entre pays déclarants. Les **charges**, elles, restent saisies par l'agriculteur : l'application ne les invente pas. De ces deux éléments elle calcule une **marge brute** — prix × rendement - charges —, qui sépare les cultures là où le rendement ne les sépare pas.

Deux cultures sur six ne sont pas couvertes par le référentiel. L'orge et le coton sont absents des données FAO exploitables. Plutôt que de supprimer un tiers des observations, nous les avons conservées en signalant explicitement l'absence de référence. L'application devra indiquer à l'utilisateur que la recommandation est moins documentée pour ces deux cultures.

Les mesures climatiques des deux sources ne sont pas comparables. La température des parcelles s'étend de 15 à 40 °C, celle du référentiel FAO est une moyenne annuelle nationale. 38 % des parcelles sont plus chaudes que la moyenne annuelle du pays le plus chaud du monde : les deux grandeurs ne mesurent pas la même chose, et nous nous sommes interdit de les croiser.

La durée jusqu'à récolte a été écartée par prudence. Nous ignorons si elle est connue au moment du semis ou seulement constatée après coup — dans le second cas, l'application ne pourrait pas la demander à l'utilisateur. Son effet sur le rendement étant nul, la conserver aurait fait courir ce risque sans aucune contrepartie. **La question reste néanmoins à poser au métier** : si la durée est bien connue à l'avance, elle pourrait être réintroduite — sans attente particulière, puisqu'elle n'explique rien.

---

6. Ce qui a été livré

Livrable	Contenu
Jeu de données consolidé	999 769 parcelles, 14 variables — source de vérité pour l'entraînement
Dictionnaire des variables	Origine, unité, signification et usage prévu de chaque colonne
Référentiel par culture	Indicateurs FAO, destinés à l'application
Panel historique FAO	56 709 observations nettoyées, pour les analyses ultérieures

Traitements appliqués : correction d'une duplication massive dans les relevés de température, révélation de valeurs manquantes dissimulées sous forme de texte, et suppression des colonnes sans information dans les fichiers sources.

Les seules suppressions, toutes documentées et chiffrées : 231 rendements de parcelles physiquement impossibles (0,02 %), 8 rendements historiques nuls ou négatifs, et 780 valeurs de pluviométrie non exploitables.

Aucune valeur n'a été inventée, et la variable à prédire n'a subi aucune modification.

Étape 2 — Modélisation : performance obtenue et leviers identifiés

Document de travail destiné à alimenter le rapport métier final. Rédigé pour un lecteur non technique.

1. Ce qui a été construit

Le jeu consolidé compte 999 769 parcelles, découpées en trois : **699 837** pour l'entraînement, **149 966** pour la validation, autant pour le test. Quatre modèles ont été entraînés sur la première partition, puis départagés sur la deuxième — qu'aucun d'eux n'avait vue pendant son apprentissage.

Modèle	Rôle	RMSE sur validation
Référence naïve	Prédit toujours le rendement moyen — sert d'étalon	1,699 t/ha
Régression linéaire régularisée	Modèle simple, effets additifs	**0,500 t/ha**
Forêt d'arbres (LightGBM)	Modèle complexe, effets non linéaires	0,501 t/ha
Forêt d'arbres (HistGBM)	Variante du précédent	0,501 t/ha

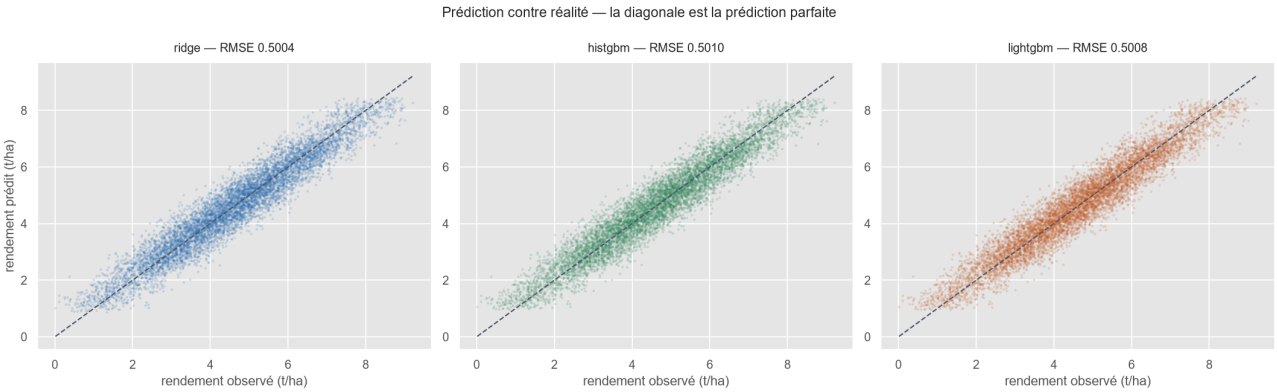
Le modèle naïve n'est pas là pour concourir : il fixe le seuil en dessous duquel un modèle ne sert à rien. Les trois autres le divisent par **3,4**.

C'est le modèle le plus simple qui gagne. Les deux modèles complexes, capables de capter des interactions subtiles, n'en trouvent aucune : ils finissent très légèrement derrière une simple addition d'effets. C'est un résultat en soi — il dit que le rendement, dans ces données, se décompose en contributions indépendantes, sans effet de seuil ni de combinaison.

2. Ce que vaut le modèle retenu

La performance annoncée est mesurée sur un troisième jeu de données, **ouvert une seule fois, après que le choix du modèle a été arrêté**. C'est ce qui rend le chiffre crédible : aucune décision n'a été prise en le regardant.

Indicateur	Valeur	Lecture métier
Variation du rendement expliquée	**91,2 %**	Le modèle capte l'essentiel de ce qui fait varier un rendement
Erreur absolue moyenne (MAE)	**0,40 t/ha**	Soit **8,6 %** du rendement moyen (4,65 t/ha)
Erreur en valeur	**80 €/ha**	À 200 €/tonne
Écart validation -> test (RMSE)	**+0,0008 t/ha**	Aucune dégradation sur le jeu tenu à l'écart. Calculé sur les valeurs non arrondies



Chaque point est une parcelle. Plus le nuage est serré autour de la diagonale, plus le modèle est juste. L'alignement est net sur toute la plage de rendement, sans zone où le modèle décrocherait.

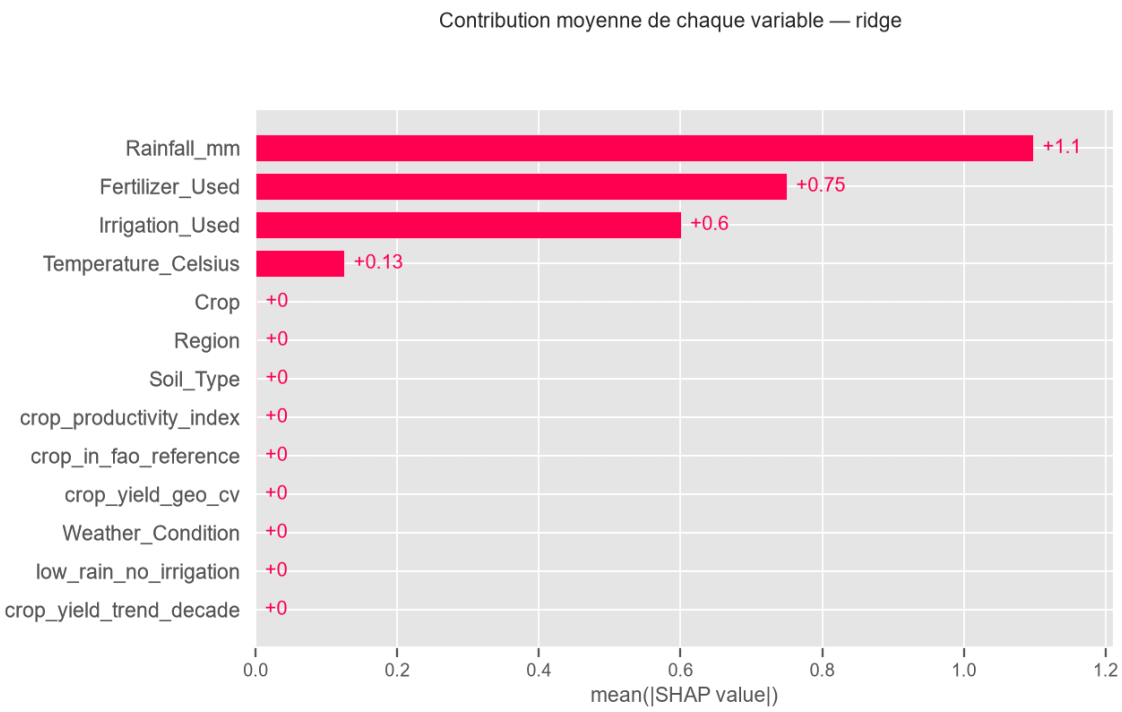
Le dernier indicateur est le plus important pour la confiance : un modèle qui excelle sur ses données d'apprentissage et s'effondre sur des données nouvelles est inutilisable. Ici l'écart est de huit dix-millièmes de tonne : **aucune dégradation n'apparaît sur le jeu tenu à l'écart**. Cela ne garantit pas la performance en production. Le test mesure la généralisation à des données issues de la **même distribution simulée** — pas à des parcelles réelles, dont rien n'assure qu'elles lui ressemblent.

---

3. Les leviers du rendement, chiffrés

C'est le résultat directement actionnable. Quatre facteurs déterminent le rendement, et **leurs effets se lisent en euros par hectare**.

Levier	Effet sur le rendement	Produit brut à 200 €/t
Fertiliser	+1,50 t/ha	**+300 €/ha**
Irriguer	+1,20 t/ha	**+240 €/ha**
Pluviométrie	+0,005 t/ha par mm	+1 €/ha par mm
Température	+0,020 t/ha par °C	+4 €/ha par °C



Contribution moyenne de chaque variable à la prédiction. Les quatre premières portent la totalité du signal ; les neuf autres sont indiscernables de zéro.

Deux lectures s'imposent :

La pluviométrie est le premier facteur, et de loin. Son effet unitaire paraît faible — 1 €/ha par millimètre — mais elle varie de 100 à 1 000 mm dans les données. D'un extrême à l'autre, cela représente **4,5 t/ha, soit 900 €/ha**. C'est le facteur dominant, et c'est aussi le seul sur lequel l'agriculteur n'a aucune prise : d'où l'importance des deux suivants.

Fertilisation et irrigation sont les deux seuls leviers pilotables, et ils sont du même ordre de grandeur : 300 et 240 €/ha de produit brut. Ce sont des décisions binaires — on fertilise ou non, on irrigue ou non — ce qui les rend directement traduisibles en conseil.

> **Réserve essentielle.** Ces montants sont du **produit brut**, pas du bénéfice. Fertiliser et irriguer coûtent, et **aucune donnée de coût n'a été fournie**. L'arbitrage « est-ce rentable ? » suppose de connaître le prix des intrants et de l'eau. Le modèle dit ce que rapporte l'action ; il ne dit pas ce qu'elle coûte.

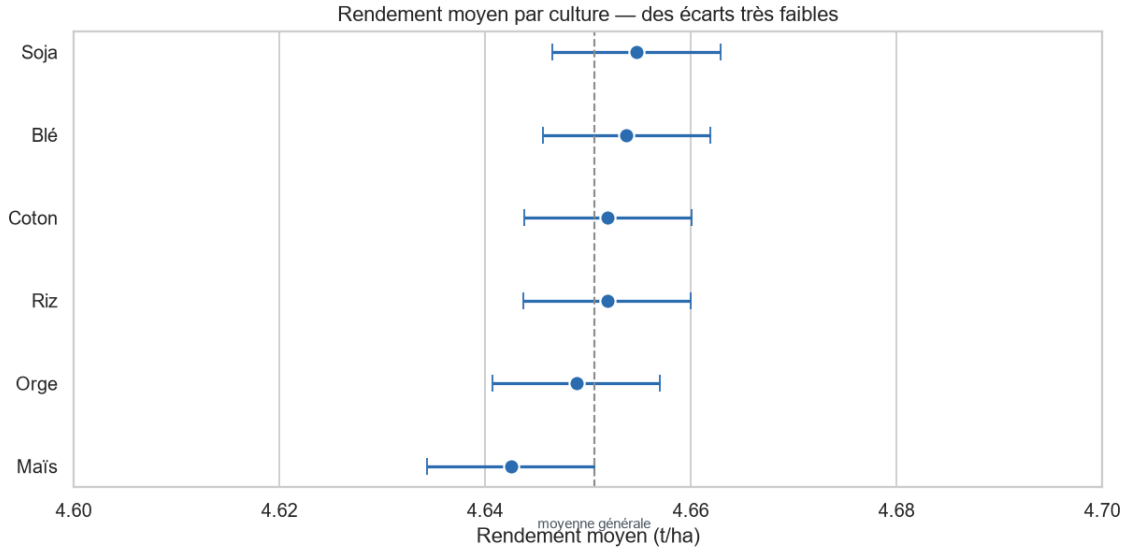
> **Suite donnée.** L'application construite aux étapes 3 et 4 demande ces coûts à l'agriculteur plutôt que de les présumer, et en déduit une marge brute. Le modèle dit toujours ce que rapporte l'action ; c'est l'utilisateur qui dit ce qu'elle lui coûte.

4. La recommandation de culture : la question est tranchée

L'étape 1 avait laissé deux voies ouvertes — injecter une référence externe pour différencier les cultures, ou renoncer au classement. **L'étape 2 tranche pour le renoncement, et le démontre.**

À conditions de parcelle rigoureusement identiques, nous avons demandé au modèle de simuler les six cultures et mesuré l'écart entre la meilleure et la pire :

Mesure	Valeur
Écart de rendement entre meilleure et pire culture	**0,0065 t/ha**
Soit, en valeur	**1,29 €/ha**
Seuil de pertinence pratique du prototype	**0,1 t/ha** (100 kg/ha)
Rapport de l'écart à ce seuil	**1 pour 15**



Attention à l'échelle : elle couvre un dixième de tonne par hectare. Les six cultures y tiennent toutes, et leurs marges d'incertitude se recouvrent.

Le gain théorique d'un changement de culture est **quinze fois plus petit que le seuil de pertinence pratique retenu pour ce prototype** — 0,1 t/ha, soit l'ordre de grandeur de ce qu'une pesée de récolte distingue réellement, une fois comptées l'humidité, les pertes au battage et l'hétérogénéité de la parcelle.

Ce seuil est une **convention de décision annoncée**, ni une norme agronomique universelle, ni un test de significativité : la significativité statistique des écarts entre cultures n'a pas été évaluée dans ce projet, et la conclusion ne repose pas sur elle.

Un second constat porte sur le modèle lui-même : **le classement est constant**. L'orge arrive en tête sur 100 % des parcelles, quelles que soient les conditions.

Ce n'est pas une découverte expérimentale, et il faut le dire : le modèle retenu étant **strictement additif**, sans aucun terme d'interaction entre la culture et les conditions de la parcelle, la contribution de la culture est une constante qui ne dépend pas du contexte. Le classement *ne pouvait pas* varier. C'est une propriété de la classe de modèle, garantie d'avance.

Correctement énoncé, le fait est d'ailleurs plus fort : un modèle additif est **structurellement incapable** d'adapter son conseil à la parcelle, et aucune quantité de données n'y changerait rien. Un conseil qui ne dépend pas de la situation de l'agriculteur n'est pas un conseil. Cela ne dit pas que les cultures réelles n'interagissent pas avec leur environnement — cela dit que ce modèle ne peut pas représenter une telle interaction.

Enfin — et c'est le résultat qui porte la conclusion — nous avons vérifié que retirer purement et simplement l'information « culture » du modèle — la variété elle-même et les quatre indicateurs de référence construits à l'étape 1 — **ne dégrade pas la prédiction**. Aucun gain prédictif n'est détecté dans nos expériences.

Ce constat ne prouve pas qu'aucun effet cultural n'existe en agriculture réelle. Il établit que **ces** données, traitées par **ce** modèle, n'en portent aucune trace exploitable.

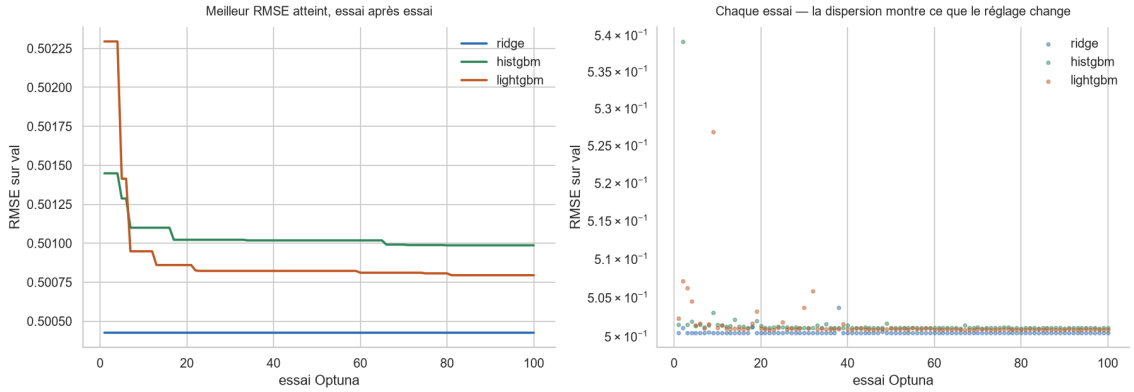
Conséquence pour l'application : elle sera présentée comme un **estimateur de rendement**, pas comme un conseiller de culture. Et le message métier n'est pas un aveu d'échec — c'est une réorientation : *dans ce modèle, la fertilisation pèse 232 fois plus que l'écart maximal entre cultures*. Les deux grandeurs sont des contributions **prédictives** estimées sur des données simulées, non des effets agronomiques démontrés : le rapport dit où se situe le signal dans le modèle, pas ce qu'un apport d'engrais produirait sur une parcelle réelle.

---

5. La démarche d'optimisation

Chaque modèle a été réglé automatiquement par une recherche d'hyperparamètres — l'équivalent du réglage fin d'un instrument. **305 expérimentations** ont été enregistrées et horodatées dans MLflow, le journal de bord du projet.

Élément	Détail
Méthode de recherche	Optuna, échantillonneur TPE — apprend des essais passés plutôt que de balayer une grille
Essais par modèle	100
Métriques suivies	Erreur (RMSE, MAE), variation expliquée (R^2), et trois indicateurs économiques
Traçabilité	Chaque essai porte l'empreinte du jeu de données, du découpage et des hypothèses de prix



À gauche, la meilleure erreur atteinte au fil des essais : les courbes s'aplatissent bien avant le centième, le budget de recherche était suffisant. À droite, la dispersion des essais montre ce que le réglage change réellement.

La reproductibilité est assurée par des graines aléatoires fixées à chaque étape — découpage des données, échantillonnage des hyperparamètres, entraînement. **Relancer la chaîne redonne exactement les mêmes résultats.**

---

6. Limites à porter au rapport final

Les données sont simulées. Le modèle retrouve presque exactement la formule qui a servi à les générer : 0,005 par millimètre de pluie, 0,020 par degré, 1,50 pour la fertilisation, 1,20 pour l'irrigation. Une telle régularité ne s'observe jamais sur des parcelles réelles. Le prototype démontre la chaîne complète, de la donnée à l'application — mais **les effets chiffrés ne sont pas des résultats agronomiques transposables**.

Le plafond de performance est atteint. Les données contiennent une part d'aléa irréductible, que nous avons mesurée à 0,500 t/ha. Le modèle atteint 0,5013 : il en est à **treize dix-millièmes**. L'essentiel du signal prédictible paraît donc capté, et les gains restants devraient être marginaux dans ce cadre expérimental. Ce n'est pas une impossibilité démontrée, mais ce que cette estimation du bruit suggère.

Produit brut, pas bénéfice. Déjà signalé à l'étape 1 et toujours valable : sans données de coût, l'application peut dire ce que rapporte une pratique, pas si elle est rentable. Obtenir ne serait-ce qu'un ordre de grandeur du prix des intrants changerait la valeur du produit.

> **Suite donnée.** Les charges sont désormais saisies par l'utilisateur, et l'application en déduit une marge brute. La limite ne disparaît pas, elle change de main : la qualité du résultat dépend maintenant de la qualité des charges que l'agriculteur renseigne.

Le modèle n'est pas encore déployable tel quel. Il attend 13 variables, alors que l'agriculteur n'en saisira que 8 : les 5 autres sont calculées à partir des premières. Tant qu'un module de préparation ne les reconstruit pas automatiquement, l'application ne peut pas interroger le modèle avec les seules saisies de l'utilisateur. **C'est le premier chantier de l'étape suivante.**

> **Suite donnée.** L'étape 3 a construit ce module — `src/agritech/preprocessing.py` — et l'a **gelé avec le modèle dans un même fichier**. La règle qui sert en production est donc, par construction, celle qui a servi à l'apprentissage : les deux ne peuvent pas diverger.

La prédiction suppose des conditions connues. Les variables climatiques décrivent une saison **observée**. L'application est un simulateur « si les conditions sont celles-ci, voici le rendement attendu » — elle ne prévoit pas la météo, et ne donne donc pas une prévision avant semis.

---

7. Ce qui a été livré

Livrable	Contenu
Modèle final	`modele_final.joblib` — 5 Ko, réponse en 3 ms, rechargement vérifié
Pipeline d'entraînement	`scripts/train.py` et le module `agritech` — préparation, optimisation, évaluation
Notebook de comparaison	Évaluation des quatre modèles, analyse des variables, décision argumentée
Journal des expérimentations	305 runs MLflow, métriques et paramètres complets
Figures	Performance, leviers, optimisation, analyse par culture

Le modèle retenu est **la régression linéaire régularisée**, choisie sur cinq critères qu'elle remporte — meilleure erreur, écart statistiquement établi avec le suivant, temps de réponse le plus court, fichier le plus léger, coefficients directement lisibles — le sixième, la régularité du comportement d'une culture à l'autre, ne départageant aucun des trois modèles.

Son avance sur le meilleur modèle complexe est pourtant **pratiquement nulle** — quelques centièmes d'euro par hectare. Ce n'est pas sa performance qui le désigne, c'est qu'il ne perd sur aucun autre critère, tout en étant le seul dont on puisse expliquer chaque prédiction sans outil d'interprétation.

Étape 3 — L'API de prédiction : contrat de service, sourçage et traçabilité

Document de travail destiné à alimenter le rapport métier final.

Les sections 1 à 7 — ce que l'application demande, ce qu'elle répond, et ce qu'elle refuse d'affirmer — se lisent sans connaissance technique : c'est la partie qui concerne l'agriculteur. Les sections 8 à 11 portent sur le déploiement et la traçabilité, et s'adressent à qui reprendra le service.

---

1. Le chantier que cette étape devait ouvrir

L'étape 2 s'était arrêtée sur un constat précis : *« Le modèle n'est pas encore déployable tel quel. Il attend 13 variables, alors que l'agriculteur n'en saisira que 8. »*

C'est le premier verrou que cette étape lève. Le second est de nature différente : un modèle qui prédit n'est pas encore un service. Il lui manque un contrat — ce qu'on peut lui demander, ce qu'il répond, et **ce qu'il refuse d'affirmer**.

Livrable	Contenu
Transformateur de dérivation	`src/agritech/preprocessing.py` — reconstruit les 5 variables calculées à partir des 8 saisies
Artefact de service	`models/modele_api.joblib` et son fichier de provenance `modele_api.json`
API	`api/` — 6 routes FastAPI, contrat Pydantic, journal des appels
Image Docker	`agritech-api` — 666 Mo, autonome, sans MLflow ni base de données
Outillage	Export et promotion du modèle, re-dérivation des prix, test de fumée du conteneur, génération des figures de ce rapport
Tests	**100 tests automatisés**, 11 secondes — et 96 % du code servi couvert, branches comprises

---

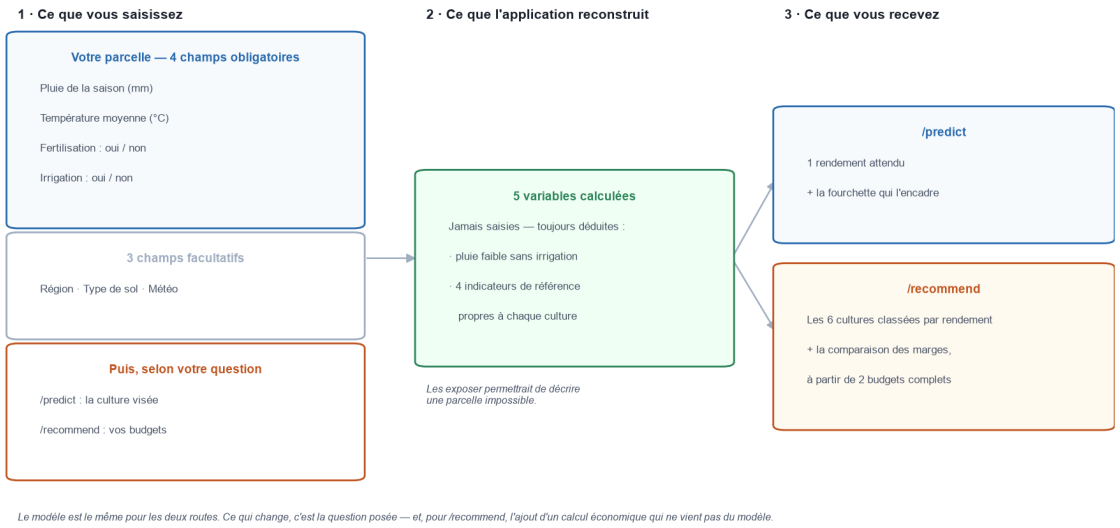
2. Le pont entre ce que l'agriculteur sait et ce que le modèle attend

Cinq des treize variables du modèle ne sont pas des saisies : elles se **calculent** à partir des autres. Quatre sont des indicateurs de référence déterminés par la seule culture ; la cinquième croise la pluviométrie et l'irrigation.

Le choix fait ici est de ne **jamais** les exposer à l'utilisateur. La raison n'est pas théorique : l'étape 2 a produit exactement le bug correspondant en changeant la culture sans mettre à jour les indicateurs associés, ce qui faussait de 24 % la mesure centrale de la recommandation. Laisser ces champs ouverts permettrait de déclarer une parcelle impossible — 800 mm de pluie, irriguée, et pourtant en situation de faible pluie sans irrigation.

La dérivation est donc un **transformateur ajusté sur les données d'entraînement**, greffé devant le modèle et gelé avec lui dans le même fichier. L'API ne réimplémente aucune règle : celle qui sert en production est, par construction, celle qui a servi à l'apprentissage. Les deux ne peuvent pas diverger.

> **Un détail qui ne se voit que dans le conteneur.** La table de correspondance mémorisée par ce transformateur a dû être forcée en type Python natif. Sans cela, la bibliothèque de données l'adosse à un format binaire tiers dès qu'il est installé sur le poste, et le fichier exporté en garde la référence : il exigerait à la relecture un paquet que l'image de l'API n'installe pas. Le défaut est invisible en développement, et fatal au déploiement.



Les cinq variables du bloc central ne sont jamais saisies : l'application les reconstruit. C'est ce qui permet d'interroger avec huit informations connues d'un agriculteur un modèle qui en attend treize.

3. Le contrat des six routes

Route	Méthode	Ce qu'elle rend	Dépend du modèle
`/predict`	POST	Le rendement d'une culture, avec son intervalle à 95 %	oui
`/recommend`	POST	Les six cultures classées, et la comparaison économique	oui
`/reference/prices`	GET	Les six prix à la production sourcés, avec leur dispersion	**non**
`/model/info`	GET	Provenance du modèle servi et valeurs acceptées	oui
`/health`	GET	Le service répond et son modèle est chargé	oui
`/logs`	GET	Les derniers appels observés, succès **et** rejets	non

3.1 `/predict` — une culture, un contexte

Cinq champs obligatoires — pluviométrie, température, fertilisation, irrigation, culture — et trois facultatifs : région, type de sol, météo. Les trois facultatifs peuvent être omis ; le modèle traite alors l'information comme absente, et la réponse le signale.

Cette omission a été **mesurée, pas supposée** : sur 5 000 parcelles, les trois champs absents déplacent la prédiction de 0,0017 t/ha en moyenne, soit 0,34 % de l'erreur du modèle. Le biais moyen est indiscernable de zéro.

En revanche, une valeur *invalide* est refusée, jamais traitée comme une omission. Sans ce garde-fou, une faute de frappe deviendrait un champ vide sans que personne ne s'en aperçoive.

La réponse ne contient **aucune traduction monétaire**. Un prix unique appliqué à six cultures n'aurait pas de sens agronomique ; la valorisation est le travail de `/recommend`, et seulement à partir des budgets de l'appelant.

3.2 `/recommend` — le classement, et ce qu'il vaut

Deux classements, de statut très différent.

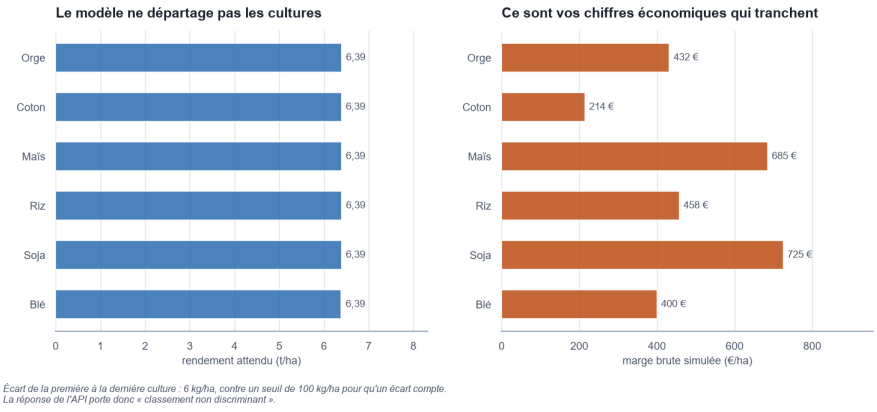
Le classement par rendement est toujours rendu, accompagné d'une note qui en donne la portée. Sur ces données, l'écart entre la première et la dernière culture est de l'ordre de 6 kg/ha, à comparer au seuil de pertinence pratique retenu pour le projet : **0,1 t/ha, soit 100 kg/ha**. La note le dit sans détour : *ce classement est exact mais sans pouvoir discriminant*.

Ce seuil est une **convention métier annoncée**, pas un seuil de significativité statistique. Il correspond à l'ordre de grandeur de ce qu'une pesée de récolte réelle distingue, une fois comptées l'humidité, les pertes au battage et l'hétérogénéité de la parcelle ; valorisé aux prix observés, il représente 21 à 51 €/ha, soit moins que la précision de n'importe quel budget prévisionnel.

> **Ce que ce seuil ne dit pas.** Que deux cultures séparées par moins de 100 kg/ha seraient « statistiquement indiscernables ». La significativité des écarts entre cultures **n'a pas été évaluée formellement** dans ce projet, et ce seuil n'en tient pas lieu : il borne la portée *pratique* d'un écart, pas sa détectabilité. C'est aussi pourquoi la note ne compare jamais l'écart à l'intervalle de prédiction — l'incertitude d'une prédiction individuelle n'est pas celle d'une différence entre deux prédictions, et les rapprocher suggérerait une égalité que rien n'établit.

Le classement économique n'apparaît qu'à partir de deux budgets complets. En dessous, aucune marge n'est comparable : le bloc reste absent plutôt que trompeur, et l'avis rendu porte le statut `undetermined` avec le motif de l'abstention.

Chaque classement économique est soumis à un **test de sensibilité au prix** : tous les prix varient simultanément de ±20 %, rendements et charges constants. Le test n'est pas tautologique — les charges ne suivant pas le facteur, l'ordre peut se renverser dès que les recettes diffèrent d'une culture à l'autre. Une paire que les trois scénarios ne départagent pas est signalée comme indistincte, et l'avis annonce alors **plusieurs cultures plutôt qu'un gagnant inventé**.



À gauche, les six cultures à conditions de parcelle identiques : six barres indiscernables. À droite, les marges calculées sur les six budgets de l'exemple documenté du contrat — de 214 à 725 €/ha. Même parcelle, même modèle : c'est l'économie qui départage, pas le rendement. Une culture dont le budget manque n'apparaît pas à droite ; l'API ne lui invente pas de marge.

3.3 `/reference/prices` — la seule route qui ne calcule rien

Elle répond même si l'artefact venait à manquer : ce sont des données externes, pas un résultat d'inférence. Son rôle est d'amorcer le champ *prix* des budgets de `/recommend` plutôt que de laisser l'utilisateur devant une case vide. La section 5 détaille d'où vient chaque chiffre.

3.4 `/logs` — les rejets comptent autant que les succès

Les appels refusés y figurent. Un rejet renseigne sur ce que les utilisateurs saisissent mal, ce qu'un journal des seuls succès tairait. C'est la raison pour laquelle l'interception se fait en amont des routes : une requête invalide n'atteint jamais le code métier.

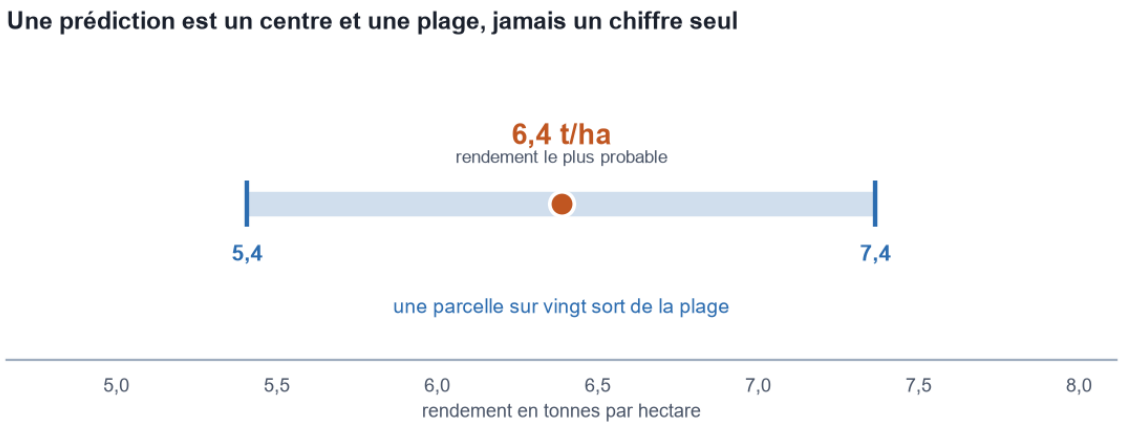
> **Deux limites assumées, répétées dans la réponse elle-même.** Le journal vit dans la mémoire du processus : il disparaît au redémarrage, et chaque worker tient le sien. Et cette route **n'est protégée par rien**, alors qu'elle expose les budgets transmis à `/recommend` — les prix et les charges de ses utilisateurs. Elle ne doit pas être ouverte publiquement en l'état.

---

4. L'intervalle de prédiction : le protocole de calibration

C'est le chiffre le plus facile à produire de travers, et le plus difficile à défendre. Le protocole retenu tient en quatre points.

Étape	Ce qui a été fait
Calibration	Quantiles empiriques des résidus, calculés sur le jeu de validation
Mesure	Couverture vérifiée sur le jeu de test, **exclu de la calibration**
Résultat	Demi-largeur **± 0,979 t/ha** — couverture mesurée **94,92 %** pour 95 % visés
Robustesse	Tenue par culture (94,73 % à 95,01 %), aux déciles extrêmes de pluie et de température, et sur les parcelles sèches sans intrants (95,27 %)



Mesuré sur les données du prototype : 94,92 % des parcelles d'un jeu de contrôle — écarté de la calibration — tombent dans leur plage. Ces données sont simulées. Cette fiabilité n'a pas été vérifiée sur des parcelles réelles, et l'intervalle devrait y être recalibré.

Le point est le rendement le plus probable, la plage celle dans laquelle il devrait tomber. Un chiffre seul se lirait comme une promesse ; c'est la plage qui porte l'information utile — et la note sous la figure dit sur quelles données cette fiabilité a été mesurée.

Le deuxième point du tableau est celui qui donne sa valeur au chiffre : la couverture n'est pas mesurée là où l'intervalle a été calibré. Un intervalle calibré et vérifié sur les mêmes données annoncerait mécaniquement le niveau visé, sans rien prouver.

Une largeur unique suffit parce que la variance des résidus est constante : l'écart-type par décile de prédiction varie de 1,5 %. La borne basse est tronquée à zéro — un rendement ne peut pas être négatif.

C'est un intervalle de prédiction, pas un intervalle de confiance. Il encadre le rendement d'*une* parcelle, pas la moyenne d'un ensemble de parcelles. Le second serait beaucoup plus étroit, et répondrait à une autre question que celle de l'agriculteur.

Hors du domaine d'entraînement — 100 à 1 000 mm de pluie, 15 à 40 °C — la réponse porte un avertissement explicite. Le modèle **extrapole linéairement et sans plafond** : rien ne borne la prédiction, et la couverture de l'intervalle n'y a pas été vérifiée. L'API avertit mais ne refuse pas : une saison plus humide que toutes celles observées reste une question légitime.

> **Réserve.** Les données sont simulées avec un bruit gaussien, dont cet intervalle mesure fidèlement la dispersion. Sur des parcelles réelles, il devrait être recalibré — et la normalité parfaite des résidus ne s'y observerait pas.

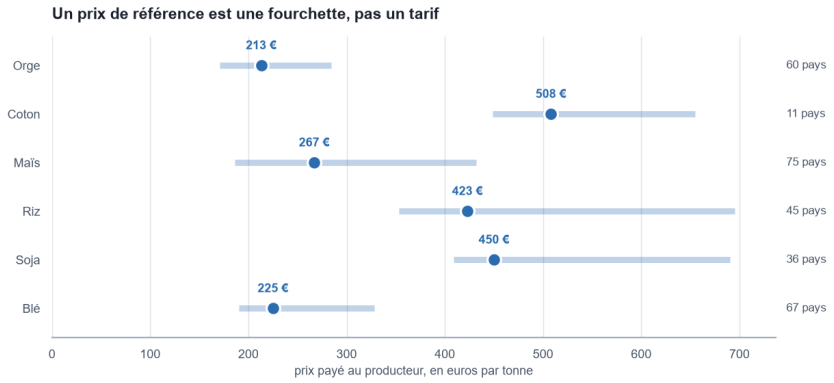
---

5. Les prix de référence : d'où vient chaque chiffre

Aucun des deux jeux de données du projet ne contient de prix. Les six valeurs servies par `/reference/prices` viennent donc de l'extérieur, et leur provenance est rendue avec elles.

Source : FAOSTAT — l'organisation statistique de la FAO, agence des Nations unies pour l'alimentation et l'agriculture. Domaine *Producer Prices*, élément 5532 « Producer Price (USD/tonne) », année 2024. Conversion en euros au taux annuel moyen BCE 2024 (1,08238 USD/EUR).

Culture	Prix médian	Fourchette interquartile	Pays déclarants
Orge	213 €/t	171 - 284	60
Coton *(graine, non égrené)*	508 €/t	449 - 655	11
Maïs	267 €/t	186 - 432	75
Riz	423 €/t	353 - 695	45
Soja	450 €/t	409 - 690	36
Blé	225 €/t	190 - 328	67



Point : prix médian. Barre : moitié centrale des pays déclarants, du 1er au 3e quartile — une dispersion géographique, ni une marge d'erreur, ni une variation dans le temps.
FAOSTAT 2024, médiane des pays déclarants, agrégats exclus. Conversion USD vers EUR au taux annuel moyen BCE 2024. Un prix à 11 pays déclarants couvre moins de terrain qu'un prix à 75 ; il n'est pas pour autant moins juste.

La barre couvre la moitié centrale des pays déclarants — une dispersion géographique, pas une marge d'erreur. Le riz s'étale de 353 à 695 €/t : lire « 423 €/t » sans voir cet étalement donnerait au chiffre une précision qu'il n'a pas.

Quatre décisions méritent d'être explicitées.

Médiane plutôt que moyenne. Quelques pays à prix très élevés — souvent de petits marchés insulaires — tireraient une moyenne vers le haut sans rien dire du prix qu'un producteur rencontre.

Les quartiles sont rendus avec la médiane, délibérément. Lire 267 €/t pour le maïs sans voir que les pays déclarants s'étalent de 186 à 432 €/t donnerait à ce chiffre une précision qu'il n'a pas. La route porte d'ailleurs sa propre réserve dans le JSON, pour le client qui consomme la donnée sans jamais ouvrir la documentation.

Le coton retenu est la graine non égrenée, pas la fibre. C'est ce qui sort de la ferme ; la fibre est un produit industriel vendu près de trois fois plus cher, et n'a rien à faire dans un budget de producteur.

La qualité des valeurs est rapportée, jamais filtrée. FAOSTAT marque chaque prix d'un drapeau : officiel, estimé, imputé, repris d'un tiers. Les **294 valeurs retenues portent toutes le drapeau « valeur officielle »** — déclarées par les pays, ni estimées ni imputées. Ce n'est pas acquis pour les années suivantes : le script de re-dérivation le **signale** au lieu d'écarter en silence, car filtrer déplacerait la médiane sans que personne ne l'ait décidé.

5.1 Des chiffres re-dérivables, pas des affirmations

Six nombres écrits à la main dans un fichier de configuration sont des affirmations. Le script `scripts/build_reference_prices.py` les rend **re-dérivables**, et peut constater qu'ils n'ont pas dérivé : exécuté en mode vérification, il recalcule tout depuis l'export FAOSTAT et sort en erreur au moindre écart. C'est ce qu'une chaîne d'intégration continue exécuterait.

> **Un piège rencontré en l'écrivant, et encodé dans le script.** Le fichier de codes produits de FAOSTAT orthographie certains produits avec un point-virgule — *« Seed cotton; unginned »* — là où le fichier de données met une virgule. Un filtre construit sur le fichier de codes ne renvoie **aucune ligne, sans lever d'erreur** : le coton disparaîtrait silencieusement du résultat, et les cinq autres cultures ne laisseraient rien soupçonner. Le script refuse désormais de continuer si l'un des six produits ne ramène rien.

---

6. Les charges : pourquoi l'API n'en invente aucune

Les prix se sourcent ; **les charges opérationnelles, non**. Aucune base publique ne donne un coût de production par hectare comparable entre pays, et transposer un barème de chambre d'agriculture française à une parcelle de localisation inconnue n'aurait pas de sens.

Cette absence est donc **assumée et portée par le contrat** : les budgets sont fournis par l'appelant, et chaque marge renvoyée est estampillée `"source": "user"`. Les exemples de la documentation — riz, blé — sont annoncés comme illustratifs.

La convention de calcul est une **marge brute simplifiée, hors aides et coproduits** : prix × rendement - charges opérationnelles. C'est l'indicateur de référence des chambres d'agriculture, dans sa forme la plus réduite. Elle ignore les charges de structure, ce qui n'est légitime que si les cultures comparées partagent le même système — vrai ici, puisque fertilisation et irrigation sont fixées pour toute la requête, donc identiques pour les six cultures.

Cette convention porte un identifiant de version, repris dans chaque réponse économique : deux résultats calculés sous des hypothèses différentes ne sont pas comparables, et il faut pouvoir le constater.

Ce que l'API établit : *telle culture présente la marge brute simulée la plus élevée parmi celles que vous avez évaluées.*

Ce qu'elle n'établit pas : *telle culture est la plus rentable pour votre exploitation.* Une marge sur une campagne n'est pas un conseil d'assolement, qui se raisonne en rotation.

---

7. Ce que l'API refuse d'affirmer

C'est la partie du contrat la moins visible et la plus structurante. Quatre refus explicites.

Aucune culture n'est déclarée agronomiquement favorable. Les six portent le statut `not_assessed`, avec son motif : le référentiel de faisabilité n'a pas été audité à ce stade. Déclarer « favorable » sans critère réellement évalué serait la seule faute possible ici — et le contrat précise que `not_assessed` ne doit pas se lire comme « favorable ».

Aucune recommandation ne sort du classement par rendement. L'étape 2 avait établi deux choses : retirer l'information « culture » du modèle ne dégrade pas la prédiction, et l'écart entre cultures reste très en deçà du seuil de pertinence pratique retenu. L'API ne revient pas sur cette conclusion par une porte dérobée — et elle ne la durcit pas davantage en équivalence statistique, qui n'a pas été évaluée.

Aucun avis quand les données ne tranchent pas. Moins de deux budgets, et le statut est `undetermined` avec le motif de l'abstention — et non le classement par rendement en guise de lot de consolation.

Aucune culture non budgétée n'est implicitement disqualifiée. Quand le classement économique ne porte que sur une partie des six, la note le dit, nomme celles qui manquent, et ajoute que *rien ne permet d'affirmer qu'elles feraient moins bien*.

Chaque réponse porte enfin un bloc `data_quality` qui rappelle que **les données d'entraînement sont simulées**, nomme la version des données, du découpage et des hypothèses économiques, et liste les avertissements déclenchés.

---

8. La traçabilité, du registre d'expériences au conteneur

Le modèle servi n'est pas celui de l'étape 2 : c'est un **artefact de service** construit à partir de lui — le transformateur de dérivation greffé devant, l'ensemble gelé. Le modèle lui-même n'est pas réentraîné.

La chaîne est la suivante, et chaque maillon est vérifié avant le suivant :

1. **Toutes les vérifications de parité passent avant la première écriture.** Un artefact qui ne prédirait pas exactement comme sa source ne doit pas atteindre le disque, et encore moins le registre. C'est le point précis où un projet d'apprentissage automatique se casse sans bruit.
2. **Publier n'est pas déclarer prêt à servir.** La version est publiée *sans alias* ; l'étiquette `champion` ne se pose qu'après les tests d'intégration, par une commande distincte.
3. **L'artefact est scellé par son empreinte SHA-256**, inscrite dans le fichier de provenance et dans le registre. Au démarrage, l'API recalcule cette empreinte et **refuse de servir** si elle ne correspond pas.

Ce sceau est le contrôle qui compte dans un conteneur. Une image fige un modèle et un fichier de provenance au moment de sa construction, et plus rien ensuite ne dit qu'ils vont ensemble : un artefact périmé, resté d'une couche antérieure ou recopié d'une autre image, servirait des prédictions en annonçant une provenance qui n'est pas la sienne. La vérification a lieu **avant** le chargement : désérialiser un fichier dont on sait déjà qu'il ne correspond pas à sa provenance n'apporterait rien et exécuterait son contenu.

> **Ce que l'empreinte n'atteste pas.** Que l'étiquette `champion` du registre désigne encore cette version. Une image est immuable, une étiquette ne l'est pas : elle a pu bouger depuis la construction. `/model/info` annonce donc la version **embarquée**, sans prétendre interroger le registre.

> **La promotion n'est pas atomique, et le script ne le prétend pas.** Elle touche deux systèmes — un fichier et un registre — qu'aucune transaction ne couvre ensemble. La propriété réellement offerte est l'**idempotence** : en cas d'échec partiel, le script le signale sans ambiguïté, et sa réexécution converge vers l'état visé **sans créer de version supplémentaire**. Il conserve par ailleurs la version précédemment étiquetée, pour permettre un retour arrière.

L'API accepte deux sources, et une seule fonction les charge : l'artefact embarqué par défaut — le cas du conteneur — ou le registre MLflow si une variable d'environnement le désigne. Dans ce second cas, un écart entre ce que sert le registre et ce que décrit la provenance **fait échouer le démarrage** : mieux vaut refuser de servir que d'estampiller une prédiction d'une provenance fausse.

Dans tous les cas, le modèle est chargé **une fois au démarrage**, et éprouvé par une prédiction témoin. Un échec se voit alors dans les journaux du déploiement, pas dans la requête d'un utilisateur.

---

9. Le conteneur

Caractéristique	Valeur
Taille de l'image	**666 Mo** décompressée sur le poste — 146 Mo une fois les couches compressées sur le registre
Démarrage	**moins de 2 s** entre `docker run` et la première réponse de `/health`, sans aucun montage externe
Compte d'exécution	utilisateur non privilégié, pas `root`
Supervision	contrôle de santé intégré, interrogeant `/health`
Dépendances installées	**25 paquets**, verrouillés à la version près

Le temps de démarrage se mesure entre `docker run` et la première réponse de `/health` : 1,6 à 1,8 seconde sur trois essais consécutifs. Le tout premier démarrage après une reconstruction demande environ 3 secondes, le temps que le système mette les couches de l'image en cache.

L'image est construite en deux temps : les dépendances sont installées dans un premier étage, et seul l'environnement résultant passe dans l'image finale. L'outillage de construction n'y figure pas.

Le conteneur est autonome. Il a été vérifié qu'il ne contient ni MLflow, ni la bibliothèque de données binaire, ni aucun des outils d'expérimentation de l'étape 2, et qu'il n'accède à aucune base de données locale. Un service de prédiction qui embarquerait sa plateforme d'expérimentation confondrait deux métiers.

Les versions ont une source unique. Le fichier de verrouillage du gestionnaire de paquets fait foi ; l'image l'installe à l'identique, et échoue si ce fichier n'est plus à jour. Le `requirements.txt` exigé par le cahier des charges en est un **export généré**, jamais édité à la main : sa régénération étant déterministe, une dérive se détecte automatiquement.

Le refus a été éprouvé, pas supposé. En substituant un faux modèle à l'intérieur du conteneur en cours de démarrage, le service **s'arrête avec un code d'erreur** en nommant les deux empreintes, celle attendue et celle constatée. C'est un test du test de fumée : il vérifie que le garde-fou se déclenche, pas seulement qu'il existe.

---

10. Ce qui a été vérifié

100 tests automatisés, exécutés en 11 secondes, portant notamment sur :

- la concordance entre les modalités annoncées par le contrat et celles réellement apprises par l'encodeur — une divergence rendrait la documentation mensongère ;
- la parité de prédiction entre l'artefact embarqué et le modèle du registre, culture par culture ;
- le comportement du sceau : artefact absent, empreinte manquante, empreinte fausse, artefact conforme ;
- l'idempotence de la promotion, et son refus en cas de provenance discordante ;
- le refus d'élire un gagnant quand trois cultures affichent la même marge : l'API nomme les trois ;
- le fait que **toutes les routes exposées sont documentées** — un test qui a d'ailleurs échoué à l'ajout de `/reference/prices`, et fait son travail ;
- la cohérence du tri sur les valeurs exactes, non arrondies : trier sur l'affichage créait des ex æquo que l'ordre de déclaration départageait en silence.

La couverture de code

Mesurée en **couverture de branches** : un test qui traverse une condition sans jamais emprunter son autre côté ne la compte pas comme couverte. C'est plus exigeant que de compter les lignes exécutées.

Module	Son rôle	Couverture
`api/schemas.py`	le contrat : ce que l'API accepte et renvoie	**100 %**
`src/agritech/config.py`	les constantes du projet	**100 %**
`api/service.py`	le moteur de calcul	**96 %**
`src/agritech/preprocessing.py`	le transformateur embarqué	**92 %**
`api/main.py`	les six routes	**90 %**
`api/journal.py`	le journal des appels	**88 %**
Ensemble du code servi		**96 %**

Le périmètre mesuré est **le code qui part dans le conteneur**. Les modules d'entraînement de l'étape 2 en sont exclus : ils s'exécutent lors d'un apprentissage, pilotés par MLflow et Optuna, et non sous les tests. Les y mêler donnerait un taux global de 76 % qui ne mesurerait ni le service ni la chaîne d'entraînement. L'exclusion est écrite et justifiée dans `pyproject.toml`, où elle est vérifiable.

Les quelques lignes non couvertes sont des **chemins de refus volontairement inatteignables** : le filet qui rattraperait une divergence entre les constantes et l'encodeur — qu'un autre test rend impossible — et la branche « le classement a un pouvoir discriminant », qui ne peut pas se produire avec ce modèle. Les couvrir reviendrait à tester des messages d'erreur.

Le rapport détaillé se régénère par `uv run pytest --cov --cov-report=html`.

S'y ajoute un **test de fumée du conteneur** de 24 contrôles, organisé en trois familles : autonomie de l'image, bon fonctionnement du service, et déclenchement effectif des refus.

---

11. Limites portées à l'étape suivante

`/logs` **n'est protégée par rien** et expose les budgets transmis à `/recommend`. L'ouvrir publiquement reviendrait à publier les données économiques de ses utilisateurs. Une authentification est à prévoir avant toute exposition réelle.

Le journal n'est pas un journal d'audit. Il vit dans la mémoire du processus, disparaît au redémarrage, et chaque worker tient le sien. Il sert à observer, pas à prouver.

Les prix sont des médianes mondiales, pas des prix français. Leur assise varie beaucoup : 11 pays déclarants pour le coton, 75 pour le maïs. Une version territorialisée serait un progrès net, et changerait la valeur de la comparaison économique.

Le référentiel de faisabilité agronomique reste à auditer. Tant qu'il ne l'est pas, les six cultures restent en `not_assessed` — le refus décrit en section 7.

Les charges opérationnelles restent à la charge de l'appelant. C'est la limite signalée depuis l'étape 1, et toujours la plus structurante : sans elles, l'API dit ce que rapporte une pratique, pas si elle est rentable.

L'étiquette du registre peut dériver sans que l'image en sache rien. Le contrôle existant est délibérément local ; une vérification de synchronisation au déploiement relèverait de la chaîne d'intégration continue, chantier de l'étape 5.

> **Suite donnée.** L'étape 5 a livré cette chaîne, et elle a tranché autrement : elle **n'interroge pas le registre**. Elle emballe l'artefact versionné, dont l'empreinte est vérifiée au démarrage du conteneur. Le rapprochement se fait en amont, dans l'orchestrateur du pipeline, qui refuse de réexporter tant que l'artefact et les partitions présentes ne décrivent pas le même découpage.

Étape 4 — L'interface Streamlit : ce que l'agriculteur voit, et ce qu'on l'empêche de croire

Document de travail destiné à alimenter le rapport métier final.

Les sections 1 à 6 décrivent l'écran et les décisions de présentation : elles se lisent sans connaissance technique. Les sections 7 à 10 portent sur l'architecture, la traçabilité et les tests, et s'adressent à qui reprendra l'application.

---

1. Ce que cette étape devait livrer

L'étape 3 s'était arrêtée sur un service qui répondait correctement — mais auquel il fallait parler en JSON. Cette étape pose une interface par-dessus, sans rien ajouter au modèle ni à l'API.

Livrable	Contenu
Application	`app/app.py` — 833 lignes, un seul fichier, deux parcours
Dépendances	`app/requirements.txt` — 40 paquets, exportés depuis `uv.lock`
Tests	`tests/test_app.py` — **44 tests**, 7 secondes, sans jamais joindre l'API
Couverture	**99 %** de `app/app.py`, branches comprises

Le cahier des charges demandait `app.py` et `requirements.txt`. Le reste — tests, panneau de traçabilité, export — relève du choix de projet, pas de l'obligation.

---

2. La règle qui gouverne tout le fichier : l'interface ne calcule rien

Aucune logique métier ne figure dans `app.py`. Pas de modèle, pas de seuil, pas de règle de dérivation. Même ce qu'on serait tenté de considérer comme du décor vient de l'API :

Ce qui s'affiche	D'où ça vient
La liste des six cultures	`accepted_values.Crop` de `/model/info`
Les bornes des curseurs — 100 à 1000 mm, 15 à 40 °C	`training_domain` de `/model/info`
Le rendement et sa fourchette	`/predict`
Le classement et ses réserves	`/recommend`
Les prix de référence	`/reference/prices`

La raison n'est pas esthétique. Recopier ces valeurs créerait un second endroit où une divergence peut s'installer — une interface qui proposerait une culture que le modèle ne connaît plus, ou un curseur qui autoriserait une saisie que l'API refusera. Si le modèle était réentraîné sur un domaine plus large, l'écran suivrait sans qu'une ligne change. Un test le vérifie explicitement.

La seule exception est assumée et signalée : les charges opérationnelles d'exemple (section 4). Ce sont des repères de saisie, pas des données, et tout ce qui les entoure le répète.

---

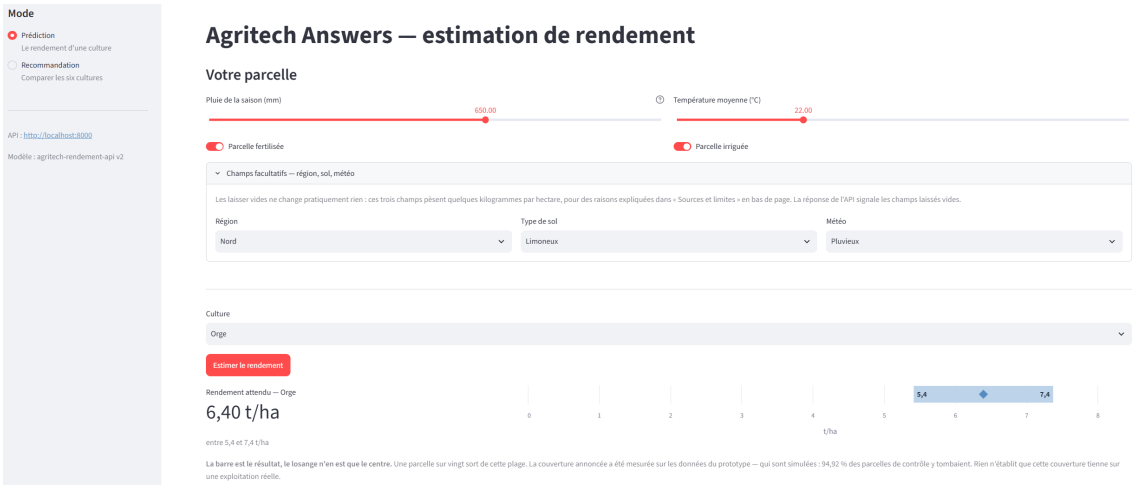
3. Les deux parcours

3.1 Prédiction — une culture, un rendement, une fourchette

L'agriculteur décrit sa parcelle — pluie de la saison, température moyenne, fertilisation, irrigation — choisit une culture, et obtient un chiffre.

Ce chiffre n'est jamais seul. À côté, une barre horizontale montre l'intervalle à 95 %, avec l'estimation posée dessus en losange. Ce choix de présentation répond à un constat simple : *« entre 5,4 et 7,4 t/ha »* écrit dans une phrase se lit comme une précision, alors qu'une barre se lit comme une incertitude. L'axe est ancré à zéro — sans cela, une plage étroite paraîtrait large.

La légende rattache la couverture annoncée à ce qui l'a mesurée : 94,92 % des parcelles de contrôle tombaient dans l'intervalle, **sur des données simulées**. Rien n'établit que cette couverture tienne sur une exploitation réelle.



Capture de l'application en fonctionnement. Le chiffre en gras est l'estimation ; la barre bleue est l'intervalle à 95 %, borné à ses deux extrémités, et le losange marque l'estimation à l'intérieur. La phrase sous le graphique rattache la couverture annoncée aux données qui l'ont mesurée.

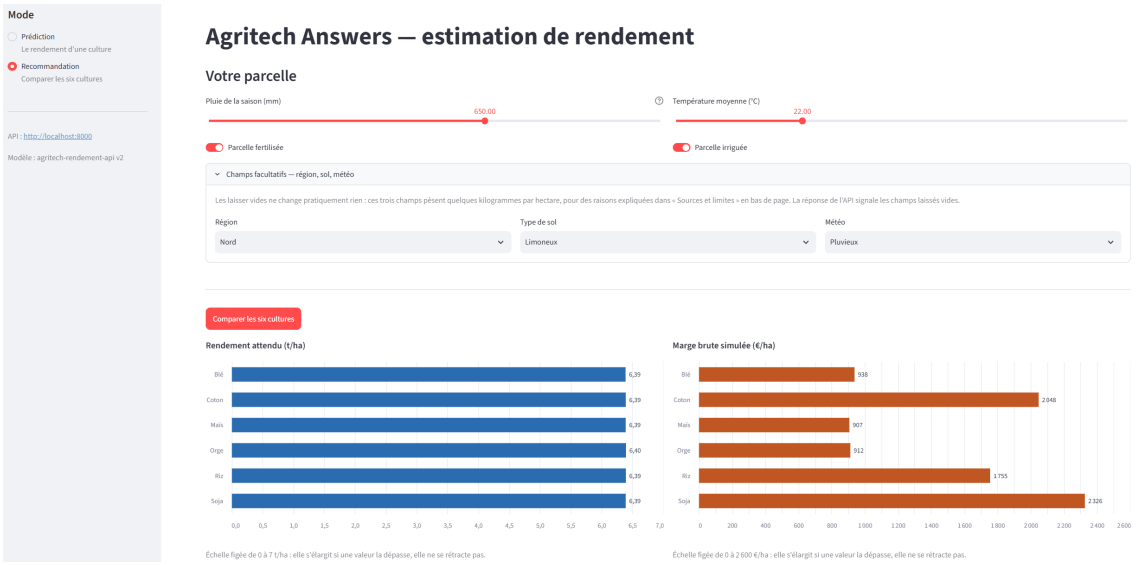
3.2 Recommandation — en deux temps, et jamais un seul

Le premier temps compare les six cultures sur le rendement attendu. Le second seulement, une fois les charges renseignées, calcule les marges.

Cette séparation n'est pas de confort. Sur les données du prototype, les six rendements prédits sont pratiquement identiques — l'écart entre la meilleure et la pire culture tient dans quelques kilogrammes. L'API le signale elle-même (`ranking_discriminative: false`) et l'écran relaie l'avertissement. **Un classement par rendement seul ne départagerait rien** ; c'est l'économie qui décide, et l'économie demande des charges.

Les deux panneaux — rendement à gauche, marge à droite — partagent un **ordre alphabétique figé**, qui ne dépend pas des valeurs. Trier chacun par sa grandeur interdirait la lecture horizontale, et en interactif les barres sauteraient à chaque charge modifiée.

L'échelle est figée elle aussi : elle s'élargit quand une valeur la dépasse, et ne se rétracte jamais. Sans cela, l'axe se recalerait sur chaque réponse et une marge en baisse réoccuperait la même largeur qu'avant — l'utilisateur verrait un changement de chiffres sans voir de changement. Le zéro reste toujours dans le cadre, et le bas descend si une marge devient négative : des charges supérieures au produit brut doivent se voir, et non disparaître en barre nulle.



*Capture de l'application en fonctionnement. **Les six barres de gauche sont égales à deux centièmes près** — 6,39 t/ha partout, 6,40 pour l'orge : le rendement ne départage rien. Les marges de droite, elles, s'étalent de 907 à 2 326 €/ha, et c'est la saisie des charges qui produit cet écart. Sous chaque graphique, la légende rappelle l'échelle retenue, pour que la capture reste lisible hors de l'écran.*

---

4. Les charges : pourquoi l'écran en propose, et pourquoi il s'en méfie

Aucun des deux jeux de données du projet ne contient de charges opérationnelles. L'interface en affiche pourtant, différentes selon la culture, pour que le tableau ne s'ouvre pas vide.

Ce sont des ordres de grandeur sans source, hiérarchisés par intensité connue : coton et riz irrigué en tête, céréales à paille en bas. Trois raisons de les différencier plutôt que de poser une valeur commune :

1. les six rendements prédits sont identiques, donc des charges égales feraient du classement des marges une simple recopie du classement des prix ;
2. des charges européennes combinées aux prix mondiaux FAO donnent des marges négatives sur trois cultures sur quatre — c'est mesuré, et consigné dans `config.py` ;
3. une valeur unique se lirait comme une moyenne de référence.

Trois garde-fous accompagnent cet affichage :

Où	Ce qui est dit
Bandeau au-dessus du tableau	« Les charges affichées sont des exemples, pas des références »
En-tête de colonne	« Charges (€/ha) · **exemple à remplacer** »
Au-dessus du classement obtenu	« Classement obtenu **sur les charges d'exemple** : aucune n'a été modifiée. Il illustre le calcul, il ne conseille rien. »

Le troisième est le plus important : l'application compare le tableau rendu au tableau proposé, et tant qu'aucune charge n'a été touchée, elle refuse de laisser passer le classement pour un résultat. Vider une case compte comme une intervention — c'est un retrait délibéré de la culture, pas un oubli, et cette culture sort de la comparaison sans recevoir une marge nulle.

Comparer deux scénarios sans tout ressaisir

Basculer la fertilisation ou l'irrigation puis relancer la comparaison **met à jour les rendements, conserve les charges déjà saisies, et recalcule les marges dans le même geste**. Les deux panneaux bougent ensemble, en un clic.

C'est précisément l'effet qu'on cherche à mesurer, et il a fallu deux essais pour le rendre : une première version effaçait les marges devenues caduques — au motif, juste, que les laisser à l'écran à côté de rendements neufs les ferait passer pour un résultat à jour. Mais l'utilisateur se retrouvait alors renvoyé au bouton « Calculer les marges » à chaque bascule, et le panneau de droite redemandait des charges déjà présentes. **Le bon comportement n'était ni de garder ni d'effacer : c'était de refaire.** L'application relance donc le calcul économique sur les charges en place, et ne redemande rien.

---

5. La marge brute : la formule, et pourquoi c'est celle-là

La formule est affichée avant le tableau de saisie, pas après :

> **Marge brute (€/ha) = Prix (€/t) × Rendement (t/ha) - Charges opérationnelles (€/ha)**

C'est l'indicateur que la **FAO** et le **RICA européen** retiennent pour comparer des cultures entre elles. Il ne soustrait que les charges opérationnelles — semences, engrais, traitements, irrigation, travaux par tiers. Les charges fixes — matériel, foncier, main-d'œuvre permanente — en sont exclues **parce qu'elles sont engagées avant le choix de la culture** : elles pèsent sur le résultat de l'exploitation, pas sur le départage des six options.

Source	Définition retenue
Glossaire FAO	marge brute = valeur de production - charges variables
NSW DPI, *About gross margin budgets*	exclut amortissement, intérêts, taxes et main-d'œuvre permanente ; *« a gross margin is not profit »*
Règlement (CEE) 79/65	marge brute standard = production brute standard - charges variables standard

Aides et coproduits ne sont pas comptés. **Le chiffre obtenu classe des cultures ; il ne dit pas ce que rapporte la parcelle.**

---

6. Les champs facultatifs : ce que pèsent vraiment les variables

Région, type de sol et météo sont proposés en saisie facultative. Ils sont bien transmis au modèle : les 28 variables de l'artefact servi les contiennent toutes, encodées en indicatrices, avec des coefficients non nuls. Ils ne pèsent simplement rien.

Les indicatrices et les booléens étant sur une échelle 0/1, leur coefficient se lit directement comme une **contribution prédictive** en tonnes par hectare :

Variable	Contribution dans ce modèle
Parcelle fertilisée	**+1,500 t/ha**
Parcelle irriguée	**+1,200 t/ha**
Pluie	**+0,005 t/ha par mm**, soit +4,5 t/ha sur la plage complète
Température	**+0,020 t/ha par °C**
Sol argileux — le plus fort des six sols	+0,0027 t/ha, soit **2,7 kg/ha**
Région Nord — la plus forte des quatre	+0,0022 t/ha, soit **2,2 kg/ha**
Météo pluvieuse — la plus forte des trois	+0,0005 t/ha, soit **0,5 kg/ha**

> **Ce que ces chiffres sont, et ce qu'ils ne sont pas.** Ce sont les contributions d'un modèle linéaire entraîné sur des **données simulées**. Elles décrivent la relation que le modèle a reproduite à partir du générateur, **pas un effet causal agronomique** : que le modèle attribue +1,5 t/ha à la fertilisation ne démontre pas qu'épandre de l'engrais sur une parcelle réelle relèverait son rendement d'autant. De même, +0,005 t/ha par mm est une contribution linéaire valable dans la plage apprise, et non une loi agronomique valable à toutes les pluviométries.

Un facteur **500 à 2 500** sépare les deux groupes. Dans ce modèle, les trois champs facultatifs ont donc un **impact numérique négligeable** sur la prédiction. Ce constat porte sur ce Ridge et sur ces données ; il ne démontre pas qu'aucun signal n'existerait dans la réalité — région, sol et météo comptent en agronomie, mais rien dans les données disponibles ne permet au modèle de l'apprendre.

Ce n'est pas un défaut du modèle, et les quatre premières lignes l'indiquent : elles reproduisent au millième la formule du générateur identifiée à l'étape 2, qui ne fait intervenir que ces quatre variables. Il n'y a donc, dans ces données-ci, rien à apprendre pour les trois autres — et rien ne changera tant que l'entraînement n'aura pas eu lieu sur des relevés réels. L'application l'écrit dans sa section « Sources et limites » plutôt que de laisser croire à une personnalisation qui n'existe pas.

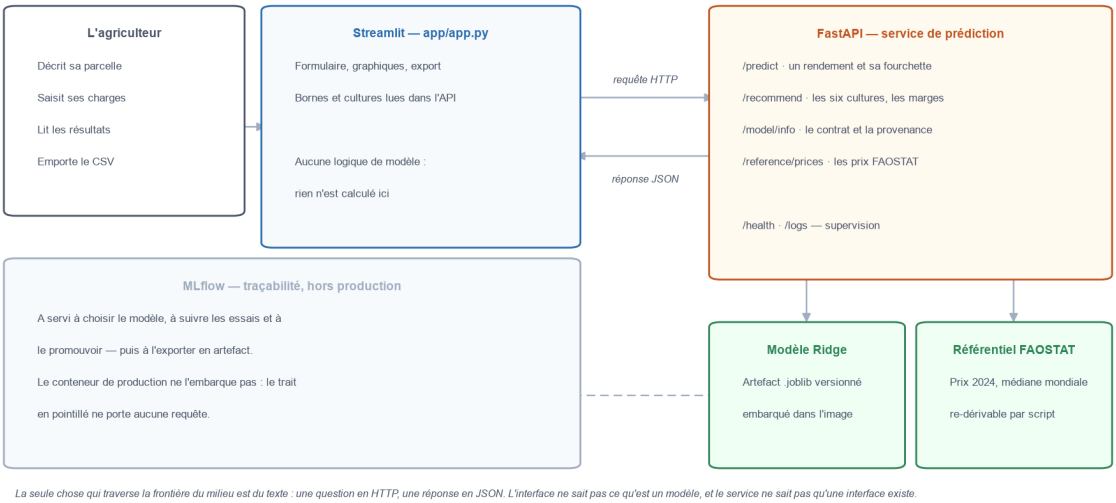
Cette lecture apporte deux constats qui dépassent la question posée.

La culture elle-même ne pèse pas davantage : `Crop_Wheat` vaut -2,7 kg/ha, `Crop_Maize` +0,4 kg/ha. C'est l'explication mécanique du `ranking_discriminative: false` renvoyé par l'API et du classement non discriminant décrit en 3.2 — si les six indicatrices de culture valent quelques kilogrammes, les six rendements prédits ne peuvent qu'être identiques. Le constat était connu par les sorties ; il se lit désormais dans les poids.

Les cinq variables dérivées non plus. `crop_productivity_index`, `crop_yield_geo_cv`, `low_rain_no_irrigation` et les autres restent toutes sous le centième de tonne. La machinerie de dérivation construite à l'étape 3 existe parce que le modèle attend treize entrées là où l'agriculteur en saisit huit — pas parce que ces cinq variables pèsent sur la prédiction de cette version. Sans elle le modèle ne se charge pas ; avec elle, il ne prédit pas mieux.

> **Une précaution de lecture.** Comparer des coefficients bruts entre variables d'échelles différentes est normalement un piège. Il n'y en a pas pour la conclusion ci-dessus : 2,7 kg/ha contre un rendement moyen de 5 t/ha font 0,05 % quelle que soit la convention. En revanche on ne peut pas en déduire que la fertilisation « pèse plus » que la pluie — à échelle comparable, la pluie l'emporte largement, puisqu'elle parcourt 4,5 t/ha entre 100 et 1000 mm.

7. L'architecture : deux processus, un seul contrat



*Le modèle et les calculs économiques restent du côté de FastAPI ; l'interface ne fait que poser des questions et afficher des réponses. **MLflow est à l'écart du flux** : il a servi à choisir, suivre et promouvoir le modèle, mais le conteneur de production ne l'embarque pas — le trait en pointillé ne porte aucune requête.*

`app.py` est organisé en deux familles de fonctions, séparées par une frontière explicite :

- **au-dessus**, les fonctions HTTP — elles ignorent qu'une interface existe et se testent sans Streamlit ;
- **en dessous**, l'affichage.

C'est cette frontière qui rend les tests possibles sans second fichier. Le groupe de dépendances `ui` est lui aussi **disjoint du groupe** `api` : le front n'importe ni FastAPI ni le modèle, et les deux pourront donc être déployés séparément.

L'adresse du service se règle par la variable d'environnement `AGRITECH_API_URL`. Sans elle, l'application vise `http://localhost:8000` ; dans un `docker compose`, elle visera `http://api:8000`. C'est le point d'entrée prévu pour l'étape 5.

Quand l'API ne répond pas, l'écran dit quoi faire plutôt que d'afficher une trace : délai dépassé, service injoignable avec la commande de démarrage, ou champ refusé — dans ce dernier cas l'application nomme le champ en cause au lieu de recracher le JSON de validation.

---

8. La traçabilité : trois niveaux qu'il serait faux de confondre

Un panneau repliable « Informations techniques du modèle » expose ce que `/model/info` déclare, en **trois blocs distincts** :

Bloc	Ce qu'il répond
Ce que l'API sert en ce moment	registre, version servie, mode de chargement, adresse du service
D'où vient cet artefact	algorithme, run MLflow, versions des données et du découpage, empreinte SHA-256, versions des bibliothèques
Ce que le registre enregistrait à l'export	alias, alias précédent, date de promotion

La séparation est le point essentiel. **Déplacer l'alias `champion` dans MLflow ne change rien à un service déjà lancé** : l'image embarque un `.joblib`, et même en mode registre l'artefact est lu une seule fois, au

démarrage. Un bouton « Relire les informations de l'API » vide le cache de cinq minutes de l'interface — il relit ce que l'API *déclare*, il ne recharge aucun modèle et ne promeut rien. La légende du bouton le dit en toutes lettres, sans quoi le panneau induirait en erreur exactement là où il prétend informer.

---

9. L'export : ce qu'on emporte

Un bouton de téléchargement produit un CSV du classement et des budgets évalués. Trois précautions y comptent plus que le tableau lui-même :

- **une culture non évaluée a une marge vide, jamais zéro** — une marge nulle se comparerait, une marge absente non ;
- **chaque colonne proposée est doublée d'une colonne d'origine**, qui distingue ce que l'application a suggéré (`FAOSTAT 2024`, `modèle`, `exemple de l'interface`) de ce que l'utilisateur a saisi ;
- **l'en-tête porte la version du modèle servi**, son run et son mode de chargement, sans quoi le fichier ne serait rattachable à rien une fois détaché de l'écran.

L'en-tête rappelle aussi la convention de marge, l'origine simulée des rendements, le caractère mondial des prix, et que le fichier documente une simulation — pas un conseil agronomique validé. Séparateur `;` et virgule décimale, pour un tableur configuré en français.

---

10. Ce qui a été vérifié

Un script Streamlit ne se teste pas en le regardant tourner : les pannes qui comptent — API éteinte, délai dépassé, saisie refusée — n'arrivent jamais pendant une démonstration. Elles arrivent chez l'utilisateur.

Les tests s'appuient sur `streamlit.testing.v1.AppTest`, le banc d'essai officiel, qui exécute réellement `app.py` et rapporte toute exception. Le faux serveur est posé sur le module `requests` lui-même, donc il vaut aussi pour le script exécuté par `AppTest`. **Aucun de ces tests ne joint l'API** : ils doivent passer en intégration continue, où aucun conteneur ne tourne.

Ce qui est couvert	Nombre
Appels HTTP et traduction des pannes	11
Parcours de prédiction et de recommandation	9
Tableau de budgets, fusion des rendements, charges d'exemple	11
Export CSV et panneau de traçabilité	6
Formules, alertes et classements	7
Total	**44 tests, 7 secondes**

Fichier	Couverture de branches
`app/app.py`	**99 %**
Total du projet (API, noyau, interface)	**97 %**, 144 tests

Trois comportements de Streamlit ont dû être établis par lecture du code source ou par l'essai, et non par supposition :

- le script utilisateur s'exécute bien avec `__name__ == "__main__"`, ce qui permet à `main()` de partir sous `streamlit run` et de rester muet à l'import ;
- `st.cache_data` **survit d'un test à l'autre** dans un même processus — une panne de l'API restait invisible jusqu'à cinq minutes, et une fixture vide désormais le cache avant chaque test ;
- `st.data_editor` n'applique les styles `pandas.Styler` **qu'aux colonnes non modifiables**, ce qui interdit de colorer la cellule des charges et explique que le signal soit porté par ce qui entoure le tableau.

---

11. Limites portées à l'étape suivante

Limite	Ce qu'elle appelle
L'interface et l'API se lancent à la main, séparément	un `docker compose` où le front vise `http://api:8000`
`app/requirements.txt` est exporté manuellement depuis `uv.lock`	un contrôle de dérive en intégration continue
Rien ne vérifie automatiquement que l'alias du registre et l'artefact servi concordent	le même contrôle, côté API
Les charges d'exemple n'ont aucune source	des références technico-économiques territorialisées, si le projet quitte le prototype
Région, sol et météo sont sans effet, la culture et les cinq variables dérivées non plus (section 6)	un entraînement sur des relevés réels — c'est la limite dont dépendent toutes les autres

> **Suite donnée.** L'étape 5 a levé la **deuxième** ligne : un contrôle de dérive compare les deux `requirements.txt` exportés à `uv.lock` à chaque poussée, et fait échouer la chaîne au moindre écart. La **troisième** l'est en partie — l'orchestrateur du pipeline compare l'empreinte des partitions citée par l'artefact servi à celle des fichiers présents, et refuse de réexporter si elles diffèrent. Les deux autres restent ouvertes, la dernière étant celle dont dépend la valeur de tout l'ensemble.

Conclusion

Ce chapitre clôt le dossier. Il reprend ce que les quatre parties établissent, ce qu'elles n'établissent pas, et ce qu'il faudrait pour que l'outil serve sur des exploitations réelles.

Ce qui a été livré

	Livrable	Caractéristique principale
1	Jeu de données consolidé	999 769 parcelles, 13 variables explicatives et la cible
	Référentiel par culture, dictionnaire des variables, panel FAO	reconstructibles par une commande
2	Modèle de prédiction	régression régularisée, 0,40 t/ha d'erreur absolue moyenne
	Journal d'expériences	305 runs, métriques et paramètres tracés
3	API de prédiction	six routes, contrat typé, journal des appels
	Artefact de service	prétraitement et modèle gelés ensemble, scellé par une empreinte
	Image conteneurisée	autonome, sans registre ni base de données
4	Interface utilisateur	deux parcours, aucune logique métier côté écran
5	Chaîne d'industrialisation	documentée dans le `README.md` du dépôt

La chaîne est complète : des six fichiers sources à une application qu'un agriculteur peut utiliser, chaque maillon se rejoue par une commande.

Ce que le projet établit

Les deux jeux de données ne se joignent pas. Aucun identifiant ne les relie : les parcelles n'ont ni pays ni année. Le référentiel FAO a donc été construit comme un enrichissement par culture, et non comme une fusion géographique. Cette contrainte a déterminé tout le reste.

Le modèle le plus simple l'emporte. Deux modèles capables de capter des interactions ne trouvent rien que la régression additive ne capte déjà. Sur ces données, le rendement se décompose en contributions indépendantes.

Les six cultures ne se départagent pas sur le rendement. À conditions de parcelle identiques, l'écart entre la meilleure et la pire vaut 6,5 kg/ha — quinze fois moins que le seuil de pertinence pratique retenu. Retirer l'information « culture » du modèle ne dégrade pas la prédiction. C'est le résultat central du projet, et il va à l'encontre de la demande initiale.

Quatre variables portent la totalité du signal : la pluviométrie d'abord, puis la fertilisation, l'irrigation et la température. Les neuf autres — type de sol, région, météo, indicateurs de référence — pèsent quelques kilogrammes par hectare, soit un facteur 500 à 2 500 d'écart.

C'est l'économie qui départage les cultures, pas l'agronomie. Les six rendements étant pratiquement identiques, seule la marge brute — prix × rendement - charges — les sépare. D'où le choix de servir des prix de référence sourcés et de demander les charges à l'utilisateur plutôt que de les inventer.

Ce que le projet n'établit pas

Aucun résultat agronomique. Les données de parcelles sont simulées : variables parfaitement uniformes, modalités rigoureusement équilibrées, relations d'une régularité qu'on n'observe jamais au champ. Le modèle retrouve d'ailleurs presque exactement la formule qui a servi à les générer. Les contributions chiffrées décrivent ce générateur, pas une agronomie.

Aucun effet causal. Que le modèle attribue +1,50 t/ha à la fertilisation ne démontre pas qu'épandre de l'engrais sur une parcelle réelle relèverait son rendement d'autant. Les données permettent ici une lecture en termes de levier — les parcelles fertilisées et non fertilisées ne diffèrent par rien d'autre, à 0,13 % près — mais cette propriété tient au caractère simulé du jeu, et ne se transposerait pas.

Aucune garantie de performance en production. Le jeu de test mesure la généralisation à des données issues de la même distribution simulée. Rien n'assure que des parcelles réelles lui ressemblent.

Aucune équivalence statistique entre cultures. Le seuil de 0,1 t/ha est une convention de décision annoncée, pas un test de significativité. Le dossier affirme que l'écart est pratiquement négligeable, jamais que les cultures sont statistiquement indiscernables.

Ce qu'il faudrait pour un usage réel

Quatre conditions, par ordre d'importance.

Des données observées. C'est la condition dont dépendent toutes les autres. Tant que l'entraînement porte sur un jeu simulé, les chiffres du modèle décrivent un générateur. Des relevés de terrain changeraient la nature même du produit — et c'est seulement alors que l'absence d'effet du sol, de la région ou de la culture pourrait être interprétée.

Des charges territorialisées. Les prix servis sont des médianes mondiales, d'assise inégale — 11 pays déclarants pour le coton, 75 pour le maïs. Les charges, elles, n'ont aucune source : l'application les demande à l'utilisateur. Des références technico-économiques régionales rendraient la comparaison économique défendable au lieu d'illustrative.

Une protection du service. La route `/logs` expose les budgets transmis par les utilisateurs — leurs prix et leurs charges — et **n'est protégée par rien**. En l'état, elle ne doit pas être ouverte publiquement. Avant toute exposition réelle, il faut une authentification, un contrôle d'accès, ou la désactivation pure et simple de cette route. C'est la limite la plus urgente de ce dossier.

Un hébergement. Aucune instance publique n'existe aujourd'hui. Les deux parcours documentés supposent une machine avec Python ou Docker ; un agriculteur ne peut donc pas se servir de l'outil sans installer quelque chose.

Ce que le prototype démontre malgré tout

Il démontre une chaîne, et la discipline qui la tient.

Le dossier montre un projet où **une demande a été contredite par les données** — on attendait un classement de cultures, les données n'en portent aucun — et où cette contradiction a été établie, mesurée, puis assumée dans le produit plutôt que masquée par un classement de bruit. L'application affiche d'ailleurs elle-même la réserve, plutôt que de laisser croire à une hiérarchie.

Il montre aussi que chaque chiffre servi est rattachable à ce qui l'a produit : l'empreinte des données, celle du découpage, la version du modèle, celle des hypothèses économiques. Une prédiction rendue aujourd'hui peut être reliée au jeu de données exact qui l'a permise.

C'est ce que doit démontrer un prototype MLOps : non pas qu'un modèle prédit bien, mais qu'on sait **ce qu'il prédit, sur quoi, et avec quelles réserves**.